

Firebox

Documentación técnica · Tienda en WhatsApp con factura electrónica (Factus v2) y cobro QR (Factus Pay)

API WARS 2026 · Universidad Distrital Francisco José de Caldas, Facultad Tecnológica · Semillero Pegasus · IEEE · Factus

Equipo Firebox: David · Dylan · Juan · Fernando Vega Benavides (líder técnico)

Octubre de 2026

Repositorio: github.com/ferdinando04/api-wars

Versión 1.0

Tabla de contenido

1. Introducción y el reto

PARTE I • SDD • Spec-Driven Development

2. Constitución del proyecto - Firebox (API WARS 2026)

3. Feature Specification: Firebox - tienda en WhatsApp con factura electrónica y QR de pago (001-tienda-whatsapp)

PARTE II • Análisis y diseño

4. DOCUMENTO DE ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE (SRS)

5. CASOS DE USO DEL SISTEMA

6. HISTORIAS DE USUARIO

7. DOCUMENTACIÓN DE INTEGRACIONES Y CONEXIONES DEL SISTEMA

PARTE III • Pruebas y evidencias

8. Evidencia Fase 0 - corrida real de punta a punta (05-oct-2026, 12:46)

9. Evidencia gráfica: panel vendedor

10. Conclusiones

11. Referencias

Introducción y el reto

Fuente: sitio oficial del evento (`apiwars.ieeeudtecono.com`) y pantallas compartidas por el equipo, 05-oct-2026.

Enunciado

"Equipos de estudiantes compiten diseñando e integrando APIs para construir soluciones funcionales en tiempo récord." APIs propuestas: las de **Factus** (instrucciones y APIs en `developers.factus.com.co`): facturación electrónica v2 y Factus Pay.

Cómo nos califican

Criterio	Peso	Qué miran	Cómo lo atiende Firebox
Integración de APIs	25 %	Uso correcto y completo de las APIs propuestas	Factus v2 (token, rangos, adquirente DIAN, emitir y validar, PDF, envío por correo) + Factus Pay (auth, crear, consultar, listar) + WhatsApp Cloud API
Funcionamiento	25 %	Cumple su propósito y funciona sin fallos	Flujo de punta a punta probado dos veces seguidas antes de presentar; idempotencia; reintentos
Calidad del código	15 %	Arquitectura, organización y documentación	SDD (constitución, spec, plan, tareas) + documentación formal + pruebas con control negativo
Innovación	15 %	Originalidad y valor	Tienda dentro de WhatsApp con factura y QR de pago en el mismo PDF; multi-tienda
Experiencia de usuario	10 %	Usabilidad, diseño y accesibilidad	Botones y listas de WhatsApp; panel claro; mensajes en español
Presentación	10 %	Claridad de la exposición y del material	Presentación + video pitch con la demo real

Cronograma (05-oct-2026)

Hora	Qué
08:00	Ingreso
09:35	Inicio del reto
13:30	Pitches finales

Hora	Qué
13:50	Cierre del evento

Qué se entrega y hasta cuándo

Modificable hasta el martes 06-oct-2026, 4:00 p. m. (UTC-5).

1. Enlace al repositorio de GitHub (público, o compartido con los jueces):
`https://github.com/ferdinando04/api-wars`.
2. Presentación (archivo o enlace).
3. Enlace al video pitch.

El código no se sube a la plataforma: debe permanecer en el repositorio.

Reglas

- Máximo 4 integrantes por equipo.
- (Pendiente de confirmar con la organización) si se permite código escrito antes del evento: por eso `referencias/` es solo guía.

Equipo

Nombre	GitHub	Línea
Fernando Vega	ferdinando04	líder técnico
Juan Aparicio	DavNor04	por definir en el issue #1
Dylan Arce	GerhardArce	por definir en el issue #1
Cuarto integrante	invitación pendiente	por definir

La historia de la demo (una sola, de punta a punta)

1. El cliente escribe a Firebox (+57 324 350 2241) desde su WhatsApp y entra a la tienda de relojes.
2. Elige un reloj, lo agrega y paga como consumidor final.
3. Recibe en el chat su factura electrónica validada (Factus v2) con la página "Paga aquí" y el QR (Factus Pay).
4. Se paga el QR con el simulador del sandbox.
5. El chat confirma "✅ Pago recibido" y el panel del vendedor muestra el pedido como pagado.

Qué hace Firebox

Convierte un número de WhatsApp en una tienda para muchos negocios: el cliente compra en el chat, recibe su factura electrónica validada por la DIAN con una página de pago con QR dentro del mismo PDF, paga, y el chat le confirma solo que la factura quedó pagada. El vendedor administra su tienda y ve sus pedidos en vivo desde un panel web. Une los dos productos del patrocinador: **Factus emite y Factus Pay cobra**, amarrados por el número de factura.

PARTE I

SDD · Spec-Driven Development

Método de trabajo: nada se programa sin especificación aprobada.

Orden: constitución, especificación, plan, modelo de datos, contratos, tareas y guía de arranque.

Constitución del proyecto - Firebox (API WARS 2026)

Reglas que NO se negocian durante la hackathon. Toda especificación, plan, tarea y revisión de código se contrasta contra este documento. Si una tarea choca con un principio, gana el principio y se replantea la tarea.

Versión: 1.1.0 · **Ratificada:** 05-oct-2026 · **Proyecto:** Firebox - tienda multi-negocio dentro de WhatsApp que emite factura electrónica (Factus) con QR de pago (Factus Pay).

Principios

I. El dinero lo calcula el código, nunca un modelo de lenguaje

- Precios, impuestos, descuentos, totales y montos de cobro se calculan con `decimal.Decimal` y redondeo `ROUND_HALF_EVEN` (bancario, el que usa la DIAN), impuesto por línea, redondear y luego sumar. Prohibido `float` para dinero.
- Si algún día se agrega IA (búsqueda en lenguaje natural), solo puede **sugerir productos**; jamás escribe un precio ni un total.
- **Invariante:** total calculado por nosotros = `total` devuelto por Factus = `amount` del cobro en Factus Pay. Si difieren, el pedido se detiene en error y no se cobra.

II. Una sola fuente de verdad: la API

- El servicio FastAPI es el **único** dueño de la base de datos y el **único** que habla con Meta, Factus y Factus Pay.
- El panel del vendedor vive en el mismo servicio (plantillas Jinja2 + HTMX) y usa las mismas funciones de servicio que la API REST: ninguna pantalla calcula dinero ni llama a Factus, Factus Pay o Meta por su cuenta.

III. Los secretos viven solo en el servidor

- Credenciales de Factus, Factus Pay y Meta: variables de entorno del servicio o columnas cifradas (Fernet) por tienda.
- Nunca en el repositorio, nunca en las plantillas del panel, nunca en el navegador, nunca en logs ni en mensajes de error devueltos al cliente.
- Cada webhook de Meta se verifica con la firma `X-Hub-Signature-256` antes de procesarse.

IV. Idempotencia en todo lo que mueve dinero o documentos fiscales

- Un mensaje de WhatsApp se procesa **una sola vez** (se registra su `id`).
- Un pedido se factura **una sola vez** (bloqueo por estado + `reference_code` único en Factus).
- Un pedido se cobra **una sola vez** (`reference_code` del cobro = número de factura; Factus Pay devuelve el mismo recaudo si se repite).

V. Una prueba solo cuenta si falla cuando se rompe lo que protege

- Toda prueba de una regla crítica (dinero, idempotencia, firma, estados) se valida con **control negativo**: romper a propósito la línea protegida, ver la prueba en rojo, restaurar. Se reporta `DETECTA el bug` o `!!! TEST INUTIL`.
- Prohibido reportar "N pruebas pasan" como prueba de calidad.

VI. Lo medido manda sobre lo documentado

- Cada contrato externo se verifica contra el sandbox antes de darlo por cierto. Ya medido el 05-oct-2026: Factus exige un `User-Agent` propio (sin él, Cloudflare responde 403); Factus Pay crea recaudos con **200 + ready + QR**, no 201/ `started` como dice su documentación; el sandbox v2 de Factus es **compartido** con otros equipos.
- Si la documentación y la realidad difieren, se anota en `docs/documentacion/07_Integraciones_Conexiones.md` con fecha.

VII. Honestidad en la demo y en el pitch

- Sandbox ≠ DIAN: las facturas de prueba no tienen validez fiscal y así se dice.
- Factus Pay sale a producción a mediados de octubre de 2026: no se presenta como "en producción".
- Ninguna cifra en el pitch que no se haya medido en la demo.

VIII. Lo mínimo que demuestra el flujo completo (YAGNI)

- Primero el camino feliz de punta a punta funcionando y desplegado; después el pulido. Cada función nueva debe responder a una historia de usuario de `specs/001-tienda-whatsapp/spec.md`.

Restricciones técnicas

Tema	Decisión
API y bot	Python 3.12+, FastAPI, SQLAlchemy 2, Pydantic 2, httpx, pytest
Panel del vendedor	En la misma app FastAPI: plantillas Jinja2 + HTMX (<code>hx-trigger="every 5s"</code>), sesión con cookie firmada httpOnly
Base de datos	SQLite en local, PostgreSQL al desplegar (cambia solo <code>DATABASE_URL</code>)
WhatsApp	Meta Cloud API oficial, Graph API v25.0, número de prueba (máx. 5 destinatarios)

Tema	Decisión
Facturación	Factus API v2 sandbox (https://api-sandbox.factus.com.co)
Cobro	Factus Pay sandbox (https://pay-api-sandbox.factus.com.co)
Despliegue	Docker (<code>docker-compose</code> : app FastAPI con API, bot y panel + db) con HTTPS público
Repositorio	<code>ferdinando04/api-wars</code> (privado), rama <code>main</code> protegida por revisión entre compañeros

Flujo de trabajo (SDD)

1. `spec.md` (qué y por qué) → revisión del equipo.
2. `plan.md` , `research.md` , `data-model.md` , `contracts/` (cómo) → revisión.
3. `tasks.md` (pasos ejecutables, con responsable) → implementación con TDD.
4. Toda tarea cierra con: prueba con control negativo + evidencia en `docs/documentacion/` (pantallazo o salida del comando).

Gobierno

- Cambiar un principio exige acuerdo del equipo y subir la versión (MAYOR si se elimina o invierte un principio, MENOR si se agrega, PARCHE si se aclara redacción).
- En caso de duda durante la hackathon decide Fernando Vega (líder técnico), dejando la razón escrita en el commit.

Feature Specification: Firebox - tienda en WhatsApp con factura electrónica y QR de pago (001-tienda-whatsapp)

Feature Branch: 001-tienda-whatsapp **Created:** 2026-10-05 **Status:** Borrador para revisión del equipo **Evento:** API WARS Hackathon 2026 - Universidad Distrital (Facultad Tecnológica) + IEEE + Factus **Input (palabras del equipo):** "Una tienda dentro de WhatsApp en la que se genere la facturación electrónica y el QR o pasarela de pago dentro de la facturación electrónica, con todo lo que tenemos en el disco y en WhatsApp."

Constitución que gobierna esta especificación: `.specify/memory/constitution.md`.
Documentación formal derivada de esta especificación: `docs/documentacion/`.

Contexto de producto

En Colombia los pequeños negocios ya venden por WhatsApp, pero lo hacen "a mano": mandan fotos sueltas, calculan el total con calculadora, piden transferencia a una llave o a Nequi, revisan el extracto para ver si pagaron y casi nunca emiten factura electrónica, aunque la DIAN la exige. **Firebox** convierte un número de WhatsApp en una tienda completa para muchos negocios a la vez:

1. El cliente compra **sin salir del chat**: catálogo con fotos, carrito y total exacto con IVA.
2. Al confirmar, recibe en el mismo chat su **factura electrónica** (Factus, validada ante la DIAN) con una **página de pago con QR Bre-B** (Factus Pay) dentro del mismo PDF.
3. Cuando paga, el chat le confirma **"factura pagada"** sin que nadie revise un extracto.
4. El vendedor ve en un **panel web** cada pedido pasar de *facturado* a *pagado* en vivo.

Es la unión natural de los dos productos del patrocinador: **Factus emite la factura y Factus Pay la cobra**, con la factura y el cobro amarrados por el mismo número.

Actores

Actor	Quién es	Cómo interactúa
Cliente	Persona que compra	Solo por WhatsApp (botones, listas, imágenes, PDF)
Vendedor	Dueño del negocio	Panel web (servido por la misma API): registra su tienda y productos, sigue pedidos
Plataforma (API)	Nuestro servicio FastAPI	Orquesta WhatsApp, Factus y Factus Pay; dueña de los datos

Actor	Quién es	Cómo interactúa
Meta Cloud API	Sistema externo	Entrega y envía los mensajes de WhatsApp
Factus API v2	Sistema externo	Emite y valida la factura electrónica ante la DIAN
Factus Pay	Sistema externo	Crea el recaudo, genera el QR Bre-B, reporta el estado del pago

User Scenarios & Testing *(mandatory)*

User Story 1 - Comprar dentro del chat (Priority: P1)

Como **cliente**, quiero ver el catálogo de una tienda, elegir productos y armar un carrito **sin salir de WhatsApp**, para comprar en el mismo lugar donde ya hablo con el negocio.

Why this priority: sin catálogo y carrito no hay compra; es el primer tramo de la demo.

Independent Test: con mensajes de WhatsApp simulados (webhook de prueba) se recorre: entrar a la tienda → abrir catálogo → ver un producto → agregar 2 unidades → ver carrito con subtotal, IVA y total correctos.

Acceptance Scenarios:

1. **Given** una tienda activa con código **RELOJES**, **When** el cliente escribe **TIENDA-RELOJES** (o abre su enlace **wa.me**), **Then** recibe un saludo con el nombre de la tienda y el botón **[Ver catálogo]**.
2. **Given** el cliente escribe cualquier texto sin código de tienda y sin conversación previa, **When** el bot responde, **Then** le muestra una lista con las tiendas activas (máximo 10 por lista; "Ver más" si hay más).
3. **Given** el cliente pulsa **[Ver catálogo]**, **When** la tienda tiene categorías, **Then** recibe una lista interactiva de categorías; al elegir una, una lista de productos (título ≤ 24 caracteres, descripción ≤ 72 con el precio).
4. **Given** el cliente elige un producto, **When** el bot responde, **Then** recibe **un solo mensaje** con la foto como encabezado, nombre, precio con IVA incluido y los botones **[Agregar]** **[Ver otro]** **[Ver carrito]**.
5. **Given** el cliente pulsa **[Agregar]**, **When** el bot pregunta la cantidad (botones **[1]** **[2]** **[3]** o un número escrito de 1 a 20), **Then** el producto queda en el carrito con esa cantidad y el bot confirma con el total parcial y los botones **[Pagar]** **[Seguir comprando]** **[Ver carrito]** (así se puede pagar sin abrir el carrito).
6. **Given** un carrito con productos, **When** el cliente pulsa **[Ver carrito]**, **Then** ve cada línea (cantidad × nombre = valor), el subtotal sin IVA, el IVA, el **total**, y los botones **[Pagar]** **[Seguir comprando]** **[Vaciar]**.
7. **Given** el cliente escribe texto libre cuando el bot esperaba un botón, **When** el texto no corresponde a ninguna opción, **Then** el bot repite las opciones válidas sin perder el carrito.

User Story 2 - Recibir la factura electrónica con su QR de pago (Priority: P1)

Como **cliente**, quiero que al confirmar mi compra me llegue al chat la **factura electrónica** con el **QR para pagarla**, para tener el soporte legal y pagar en el mismo paso.

Why this priority: es el corazón del reto (Factus + Factus Pay) y lo que diferencia el proyecto.

Independent Test: con un carrito armado se confirma la compra contra los sandbox reales de Factus v2 y Factus Pay, y se verifica: factura validada (`is_validated = true`), número `SETP...`, CUFE, recaudo `ready` con QR, y un PDF enviado que contiene la factura y la página de pago.

Acceptance Scenarios:

1. **Given** un carrito con total entre \$10.000 y \$12.000.000, **When** el cliente pulsa **[Pagar]**, **Then** el bot pregunta **[Factura a mi nombre]** **[Consumidor final]**.
2. **Given** el cliente elige **[Factura a mi nombre]**, **When** escribe tipo y número de documento, **Then** la plataforma consulta el adquirente en la DIAN vía Factus (`GET /v2/dian/acquirer`) y, si existe, pregunta *"¿Eres «nombre», «correo enmascarado»?"* **[Sí]** **[Corregir]**.
3. **Given** la consulta a la DIAN responde 404 (adquirente no encontrado), **When** el bot lo detecta, **Then** pide nombre completo y correo, uno por mensaje, y los valida (correo con formato válido).
4. **Given** el cliente elige **[Consumidor final]**, **When** continúa, **Then** no se le piden datos personales y la factura usa el cliente genérico definido por Factus.
5. **Given** los datos completos, **When** el bot muestra el resumen (productos, total, datos de factura) y el cliente pulsa **[Confirmar compra]**, **Then:** - se emite la factura en Factus v2 (`POST /v2/bills/validate`) con `reference_code` único, - se crea el recaudo en Factus Pay (`POST /v1/collections`) con `reference_code` = **número de la factura** y `amount` = **total de la factura**, - el cliente recibe, en este orden: (a) un texto "Tu factura SETP.. está lista", (b) **un PDF** con la factura de Factus y una página final "Paga aquí" con el QR, el monto, la referencia y el vencimiento (24 h), (c) **la imagen del QR** sola, (d) un texto con instrucciones de pago.
6. **Given** el total del carrito es menor de \$10.000 o mayor de \$12.000.000, **When** el cliente pulsa **[Pagar]**, **Then** el bot explica el rango permitido por Factus Pay y **no** emite factura.
7. **Given** Factus rechaza la factura (HTTP 422), **When** la plataforma recibe el error, **Then** el cliente recibe un mensaje amable ("no pudimos emitir tu factura, el negocio ya fue avisado"), el pedido queda en `error_factura` con el detalle de Factus y **no** se crea cobro.

User Story 3 - Pagar y recibir la confirmación automática (Priority: P1)

Como **cliente**, quiero que cuando pague el QR el chat me confirme solo que mi factura quedó pagada, para no tener que mandar comprobantes.

Why this priority: cierra el ciclo de la demo (compra → factura → pago → confirmación).

Independent Test: con un recaudo `ready`, se simula el pago en el simulador del sandbox de Factus Pay y se verifica que en ≤ 10 s el pedido pasa a `pagado` y el cliente recibe el mensaje de confirmación.

Acceptance Scenarios:

1. **Given** un pedido en `cobro_pendiente`, **When** el vigilante de pagos consulta `GET /v1/collections/{referencia}` y el estado es `paid`, **Then** el pedido pasa a `pagado`, se registra la hora, y el cliente recibe "✅ Pago recibido. Tu factura SETP.. quedó pagada. ¡Gracias por comprar en «tienda»!".
2. **Given** un pedido en `cobro_pendiente` con más de 24 h desde la creación del QR, **When** el vigilante lo revisa, **Then** el pedido pasa a `vencido` y se deja de consultar.
3. **Given** el cliente escribe "ya pagué" (o pulsa [Ya pagué]), **When** el pedido sigue pendiente, **Then** la plataforma consulta el estado en ese momento y responde con el resultado real (pagado o "aún no vemos el pago, puede tardar unos segundos").
4. **Given** el vigilante ya marcó un pedido como pagado, **When** vuelve a ver `paid` en la siguiente consulta, **Then no** envía un segundo mensaje de confirmación.

User Story 4 - Registrar mi tienda y mi catálogo (Priority: P1)

Como **vendedor**, quiero registrar mi negocio y mis productos en un panel web, para tener mi tienda en WhatsApp sin programar.

Why this priority: es lo que convierte el proyecto en plataforma multi-tienda; sin tiendas no hay catálogo.

Independent Test: desde el panel desplegado, un vendedor nuevo crea la tienda, carga 3 productos (uno con IVA 19 %, uno con 5 % y uno excluido) con foto, y obtiene su enlace `wa.me`; al abrir el enlace, el bot muestra esa tienda.

Acceptance Scenarios:

1. **Given** el formulario de registro, **When** el vendedor ingresa nombre del negocio, NIT, correo, código corto de tienda (único, solo letras/números, 3-20) y sus credenciales de Factus Pay, **Then** la API crea la tienda, cifra las credenciales, y devuelve un token de acceso del panel y el enlace `https://wa.me/<número>?text=TIENDA-<CÓDIGO>`.
2. **Given** las credenciales de Factus Pay ingresadas son inválidas, **When** la API intenta autenticarse con ellas, **Then** el registro se rechaza con el mensaje "No pudimos conectar con tu cuenta de Factus Pay" y no se guarda nada.
3. **Given** una tienda creada, **When** el vendedor agrega un producto con nombre, categoría, precio **sin IVA**, tarifa de IVA (19 %, 5 %, 0 % exento o excluido) y una foto (JPG/PNG ≤ 5 MB), **Then** el producto queda activo y el panel muestra el precio con IVA calculado por la API.
4. **Given** un producto activo, **When** el vendedor lo desactiva, **Then** deja de aparecer en el catálogo de WhatsApp, pero los pedidos anteriores conservan su copia fija del producto.

5. **Given** el vendedor tiene un archivo CSV con columnas `nombre`, `categoria`, `precio_sin_iva`, `iva`, `foto_url`, **When** lo carga, **Then** la API crea los productos válidos y reporta fila por fila los rechazados con el motivo. (P2 dentro de esta historia.)
-

User Story 5 - Seguir pedidos, facturas y pagos en vivo (Priority: P2)

Como **vendedor**, quiero ver mis pedidos y cómo pasan de facturado a pagado sin recargar la página, para saber qué despachar.

Why this priority: es el momento "wow" del panel en la demo, pero la compra funciona sin él.

Independent Test: con el panel abierto, se hace una compra por WhatsApp y se simula el pago; el panel muestra el pedido nuevo y su cambio a `pagado` en ≤ 10 s sin recargar.

Acceptance Scenarios:

1. **Given** el panel de pedidos abierto, **When** entra un pedido nuevo, **Then** aparece en ≤ 10 s con: fecha, cliente (o "Consumidor final"), total, estado, número de factura y enlace al PDF.
 2. **Given** un pedido, **When** el vendedor abre su detalle, **Then** ve la línea de tiempo de eventos (mensaje recibido, factura emitida, cobro creado, pago recibido) con hora y, si hubo error, el detalle devuelto por Factus o Factus Pay.
 3. **Given** un pedido en `error_cobro` (factura emitida pero el cobro falló), **When** el vendedor pulsa **[Reintentar cobro]**, **Then** la API vuelve a crear el recaudo con la misma referencia y, si funciona, envía el QR al cliente.
 4. **Given** el panel, **When** el vendedor mira el resumen del día, **Then** ve cuántos pedidos, cuánto facturado y cuánto pagado (sumas calculadas por la API).
-

User Story 6 - Elegir cómo recibir el pedido (Priority: P2)

Como **cliente**, quiero indicar si recojo en la tienda o me lo envían, para que el negocio sepa qué hacer con mi compra.

Independent Test: en el flujo de compra se elige **[Envío a domicilio]**, se escribe una dirección, y el panel muestra esa dirección en el pedido.

Acceptance Scenarios:

1. **Given** el cliente pulsó **[Pagar]** y dio sus datos de factura, **When** el bot pregunta **[Recojo en tienda]** **[Envío a domicilio]**, **Then** con "Envío" pide la dirección en un mensaje de texto (5-200 caracteres) y la guarda en el pedido.
 2. **Given** se eligió envío, **When** se emite la factura, **Then** el costo de envío **no** se suma (fuera de alcance v1; el vendedor coordina el envío por fuera) y el resumen lo dice explícitamente.
-

User Story 7 - Buscar productos escribiendo (Priority: P3, opcional)

Como **cliente**, quiero escribir lo que busco ("reloj dorado para mujer") y recibir productos parecidos, para no recorrer todo el catálogo.

Independent Test: con la tienda de relojes, escribir "dorado mujer" devuelve una lista con productos cuyo nombre, color o categoría coinciden.

Acceptance Scenarios:

1. **Given** el cliente está dentro de una tienda, **When** escribe un texto que no es un comando, **Then** la API busca coincidencias (primero por texto; IA solo si hay tiempo) y responde con una lista de máximo 10 productos o "no encontré nada, mira el catálogo".
2. **Given** se usa IA para interpretar el texto, **When** devuelve resultados, **Then** los precios mostrados salen **siempre** de la base de datos, nunca del modelo (Constitución, principio I).

Edge Cases

- **Mensaje repetido por Meta:** Meta reintenta webhooks; el mismo `message.id` se ignora la segunda vez (no crea pedido ni respuesta doble).
- **Número no autorizado:** el número de prueba de Meta solo escribe a 5 destinatarios registrados; un envío a otro número devuelve el error `131030`. La API lo registra y el panel muestra "destinatario no autorizado en modo de prueba".
- **Ventana de 24 h:** si pasan más de 24 h desde el último mensaje del cliente, WhatsApp no deja enviar texto libre (requiere plantilla). Como el QR también vence en 24 h, la confirmación de pago casi siempre cae dentro de la ventana; si no, se registra en el panel y no se reintenta.
- **Precio cambia mientras el cliente compra:** el carrito guarda el producto, pero al **confirmar** se recalcula con el precio vigente y se muestra el resumen actualizado antes de facturar. El pedido guarda una **copia fija** de nombre, precio e IVA.
- **Producto desactivado con el carrito abierto:** al pulsar **[Pagar]** se retira del carrito y se avisa.
- **Carrito vacío:** **[Pagar]** con carrito vacío responde "tu carrito está vacío" y muestra **[Ver catálogo]**.
- **Monto fuera de rango de Factus Pay:** menor de \$10.000 o mayor de \$12.000.000 → no se factura (historia 2, escenario 6).
- **Factus caído o lento:** 3 reintentos con espera creciente; si sigue fallando → `error_factura`, mensaje amable, aviso en el panel.
- **Factura emitida pero Factus Pay falla:** el pedido queda en `error_cobro`; se envía la factura sin QR y el panel ofrece **[Reintentar cobro]**.
- **Sandbox v2 compartido:** otros equipos usan la misma cuenta de Factus; nuestra `reference_code` de factura lleva prefijo único `FB-<TIENDA>-<id_pedido>` para no chocar.
- **Doble clic en [Confirmar compra]:** el segundo clic encuentra el pedido ya en `facturando` / `facturado` y responde con el estado, sin facturar de nuevo.

- **Cliente cambia de tienda a mitad de compra:** escribir `TIENDA-<OTRO>` pregunta si quiere abandonar el carrito actual `[Sí, cambiar] [No, seguir]`.
- **Foto de producto inválida o > 5 MB:** el panel la rechaza al subirla; si una foto falla al enviarse por WhatsApp, se envía el producto como texto con los mismos botones.
- **Adquirente no encontrado en la DIAN:** se piden nombre y correo a mano (historia 2, escenario 3).
- **Cliente escribe "cancelar" antes de confirmar:** se vacía el carrito y se vuelve al saludo de la tienda.

Requirements *(mandatory)*

Functional Requirements

Canal WhatsApp

- **FR-001:** La API MUST exponer `GET /webhooks/whatsapp` para la verificación de Meta (`hub.mode` , `hub.verify_token` , `hub.challenge`) y `POST /webhooks/whatsapp` para recibir mensajes.
- **FR-002:** La API MUST verificar la firma `X-Hub-Signature-256` (HMAC-SHA256 con el App Secret) de cada POST y rechazar con 401 las firmas inválidas.
- **FR-003:** La API MUST responder 200 a Meta en < 2 s y procesar el mensaje en segundo plano.
- **FR-004:** La API MUST registrar el `id` de cada mensaje entrante y descartar duplicados.
- **FR-005:** La API MUST poder enviar: texto, lista interactiva, botones de respuesta (máx. 3, con encabezado de imagen opcional), imagen y documento PDF, respetando los límites de Meta (lista: 10 filas, título 24, descripción 72, botón 20 caracteres).
- **FR-006:** El envío por WhatsApp MUST estar aislado en un adaptador de canal con la interfaz `enviar_texto` , `enviar_lista` , `enviar_botones` , `enviar_imagen` , `enviar_documento` , para poder cambiar de proveedor sin tocar la conversación.

Conversación

- **FR-010:** La conversación MUST modelarse como una máquina de estados determinista por número de WhatsApp: `inicio` → `en_tienda` → `viendo_catalogo` → `viendo_producto` → `eligiendo_cantidad` → `en_carrito` → `datos_factura` → `entrega` → `confirmando` → `esperando_pago` → `cerrada` .
- **FR-011:** Cada estado MUST aceptar solo sus entradas válidas y, ante cualquier otra, repetir las opciones sin perder datos.
- **FR-012:** Las palabras `cancelar` , `menu` y `carrito` MUST funcionar en cualquier estado anterior a `confirmando` .
- **FR-013:** El código de tienda MUST reconocerse con el patrón `TIENDA-<CÓDIGO>` (mayúsculas o minúsculas) en cualquier mensaje.

Catálogo y carrito

- **FR-020:** El catálogo MUST mostrar solo productos activos de la tienda actual, agrupados por categoría, paginados de a 10.
- **FR-021:** El carrito MUST guardar por línea: producto, cantidad (1-20) y precio vigente; y MUST recalcular totales en cada cambio.
- **FR-022:** Los totales MUST calcularse por línea: `base = precio_sin_iva × cantidad`, `iva = round_half_even(base × tarifa / 100, 2)`; `subtotal = Σ base`, `iva_total = Σ iva`, `total = subtotal + iva_total`. Las líneas excluidas no suman IVA y no se reportan como base gravable.

Facturación (Factus v2)

- **FR-030:** La API MUST autenticarse con OAuth2 `password` (form-urlencoded) y renovar con `refresh_token` antes de que venza (`expires_in` = 3600 s).
- **FR-031:** Toda llamada a Factus MUST enviar un `User-Agent` propio (`Firebox/1.0`); sin él Cloudflare responde 403.
- **FR-032:** La factura MUST emitirse con `POST /v2/bills/validate`, `document: "01"`, `operation_type: "10"`, `numbering_range_id` del rango de Factura de Venta activo, `reference_code` = `FB-<TIENDA>-<id pedido>`, ítems con precio **sin impuestos** en string con 2 decimales, `unit_measure_code: "94"`, `standard_code: "999"`, `taxes: [{code: "01", rate}]` (o `is_excluded: true` para excluidos), y `payment_details` con `payment_form: "2"` (crédito), `payment_method_code: "47"` (transferencia), `amount` = total y `due_date` = hoy + 1 día. Si el sandbox rechaza crédito con consumidor final, se usa `payment_form: "1"` (contado) con el mismo método, y la diferencia se anota en `docs/documentacion/07_Integraciones_Conexiones.md` §11.
- **FR-033:** Para consumidor final, el `customer` MUST ser exactamente el del ejemplo oficial de Factus v2 ("Con consumidor final").
- **FR-034:** La API MUST consultar `GET /v2/dian/acquirer?`
`identification_document_code=&identification_number=` para autocompletar nombre y correo del cliente, tratando 404 como "no encontrado".
- **FR-035:** Tras emitir, la API MUST guardar número, CUFE, `is_validated`, `errors` (notificaciones DIAN) y `total` devuelto, y MUST comprobar que el `total` de Factus = total calculado (Constitución I); si difiere → `error_factura`.
- **FR-036:** La API MUST descargar el PDF con `GET /v2/bills/{numero}/download-pdf` (respuesta `{file_name, pdf_base_64_encoded}`).

Cobro (Factus Pay)

- **FR-040:** La API MUST autenticarse en Factus Pay (`POST /auth`) una vez por tienda y guardar el token cifrado (no vence).
- **FR-041:** La API MUST crear el recaudo con `POST /v1/collections` `{reference_code: <número de factura>, amount: <total>}` y aceptar tanto 200 como 201; si el QR llega `null`, MUST consultarlo con `GET /v1/collections/{referencia}` hasta 3 veces.
- **FR-042:** La API MUST validar antes de facturar que `10.000 ≤ total ≤ 12.000.000`.

- **FR-043:** El vigilante de pagos MUST consultar cada 5 s los pedidos en `cobro_pendiente` y pasar a `pagado` al ver `paid`, o a `vencido` a las 24 h. Límite: 80 llamadas/min por cuenta de Factus Pay; si una cuenta tiene más de 6 cobros pendientes, el intervalo de cada uno se alarga a `ceil(pendientes × 60 / 70)` segundos (deja margen de 10 llamadas/min para crear cobros y para [Ya pagué]), priorizando los cobros creados hace menos de 10 minutos.
- **FR-044:** Las transiciones a `pagado` MUST ser idempotentes: un solo mensaje de confirmación por pedido.

Documento factura + pago

- **FR-050:** La API MUST generar un único PDF = PDF de Factus + una página final "Paga aquí" con: nombre de la tienda, número de factura, total, QR (imagen del recaudo), referencia, fecha y hora de vencimiento, e instrucciones Bre-B.
- **FR-051:** La API MUST enviar ese PDF como documento de WhatsApp con nombre `Factura <número>.pdf` y, aparte, la imagen PNG del QR.

Panel y API REST propia

- **FR-060:** La API MUST exponer endpoints REST documentados en OpenAPI (`/docs`) para: registro de tienda, inicio de sesión del vendedor, CRUD de productos, carga CSV, lista y detalle de pedidos, reintento de cobro y resumen del día.
- **FR-061:** Todo endpoint del panel MUST exigir `Authorization: Bearer <token de tienda>` y devolver solo datos de esa tienda.
- **FR-062:** El panel MUST servirse desde la misma app FastAPI (Jinja2 + HTMX), usar sesión con cookie firmada `httpOnly`, y refrescar la lista de pedidos cada 5 s (`hx-trigger="every 5s"`) sin recargar la página.

Multi-tienda y seguridad

- **FR-070:** Cada tienda MUST tener su propio código, credenciales de Factus Pay cifradas y catálogo; ninguna consulta puede mezclar datos de dos tiendas.
- **FR-071:** Las credenciales MUST cifrarse con una llave maestra de entorno (`FERNET_KEY`) y nunca devolverse por la API.
- **FR-072:** Los logs MUST omitir tokens, contraseñas y el número completo de documento del cliente (enmascarado).

Trazabilidad

- **FR-080:** Cada cambio de estado de un pedido MUST registrar un evento con hora, tipo y detalle (sin secretos).

Key Entities

- **Tienda:** negocio vendedor. Código único, nombre, NIT, correo, logo, credenciales Factus Pay (cifradas), token del panel (hash), activa.

- **Producto:** pertenece a una tienda. Categoría, nombre, descripción corta, precio sin IVA, tarifa de IVA (19/5/0/excluido), unidad (94), foto, activo.
- **Conversación:** una por número de WhatsApp. Tienda actual, estado de la máquina, carrito (líneas), datos de factura, tipo de entrega, dirección, último mensaje.
- **Pedido:** compra confirmada. Tienda, número de WhatsApp, copia fija de líneas (nombre, cantidad, precio, IVA), subtotal, IVA, total, datos de factura, entrega, estado, número de factura, CUFE, referencia del cobro, QR, ruta del PDF, fechas.
- **Evento:** bitácora de un pedido (tipo, detalle, hora).
- **MensajeProcesado:** id de mensaje de Meta ya atendido (idempotencia).

Estados del pedido: confirmado → facturando → facturado → cobro_pendiente → pagado ; salidas de error: error_factura , error_cobro , vencido .

Success Criteria *(mandatory)*

Measurable Outcomes

- **SC-001:** Un cliente completa la compra de un producto como consumidor final (desde "TIENDA-RELOJES" hasta recibir factura + QR) en **≤ 3 minutos** y **≤ 15 toques** (cada lista de WhatsApp cuesta 2: abrir y elegir), medido en la demo.
- **SC-002:** Desde **[Confirmar compra]** hasta recibir el PDF y el QR pasan **≤ 15 s** (sandbox), medido en 5 compras seguidas.
- **SC-003:** Desde que se simula el pago hasta el mensaje "✅ Pago recibido" pasan **≤ 10 s**.
- **SC-004:** En 20 compras de prueba con IVA mezclado (19 %, 5 %, excluido), **0 diferencias** entre total calculado, total de Factus y monto del recaudo.
- **SC-005:** Reenviar 10 veces el mismo webhook produce **1** pedido, **1** factura y **1** recaudo.
- **SC-006:** Un vendedor nuevo registra su tienda y 3 productos en **≤ 5 minutos** desde el panel desplegado.
- **SC-007:** La prueba de punta a punta pasa **2 veces seguidas** sobre la URL desplegada antes de presentar.
- **SC-008:** Cada prueba de reglas críticas tiene control negativo registrado (DETECTA el bug).

Assumptions

- La demo corre en **sandbox**: facturas de Factus sin validez fiscal; el QR se paga con el **simulador** de Factus Pay.
- Todas las tiendas de la demo facturan con la **misma cuenta sandbox v2** de Factus (la que entregó la organización); en producción cada tienda tendría la suya. El modelo de datos ya separa credenciales por tienda.

- Factus Pay: la tienda de relojes usa la cuenta del equipo y una segunda tienda usa la cuenta personal de Fernando, para demostrar que el dinero de cada negocio va a su propia cuenta.
- WhatsApp: número de prueba de Meta con hasta 5 destinatarios registrados (los 4 integrantes + 1 jurado).
- Moneda COP; impuestos soportados: IVA 19 %, 5 %, 0 % (exento) y excluido. Sin retenciones ni INC en v1.
- La tienda de ejemplo carga el catálogo real de NovaMarket (`catalogo_tienda.json` , 74 relojes con fotos).
- El equipo trabaja en español; mensajes del bot en español colombiano con tuteo.

Out of Scope (v1)

- Costos y logística de envío, inventario/stock, devoluciones.
- Anulación automática con nota crédito cuando el QR vence (queda para v2; en v1 el pedido pasa a `vencido`).
- Pagos con tarjeta u otras pasarelas; múltiples monedas.
- Mensajes iniciados por la tienda (requieren plantillas aprobadas por Meta).
- Registro de nuevos números de WhatsApp por tienda (v1: un número para todas, con código de tienda).
- App móvil; roles múltiples dentro de una tienda.

PARTE II

Análisis y diseño

Documentación formal derivada de la especificación: requisitos, casos de uso, historias e integraciones.

DOCUMENTO DE ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE (SRS)

PROYECTO: Firebox - tienda multi-negocio en WhatsApp con factura electrónica (Factus) y QR de pago (Factus Pay) **EQUIPO:** Equipo API WARS (integrantes en `RET0.md`) · Líder técnico: Fernando Vega Benavides **EVENTO:** API WARS Hackathon 2026 - Universidad Distrital, Facultad Tecnológica · **FECHA:** 05-oct-2026 · **VERSIÓN:** 1.0 **FUENTE:** `specs/001-tienda-whatsapp/spec.md` (especificación SDD). Si hay diferencia, se corrige aquí para que coincida con la spec.

1. INTRODUCCIÓN

1.1 Propósito

Definir los requisitos funcionales y no funcionales de Firebox, una plataforma que permite a varios negocios vender por WhatsApp, emitir la factura electrónica de cada venta y cobrarla con un QR Bre-B, para que el equipo de desarrollo, los evaluadores de la hackathon y cualquier lector técnico entiendan qué hace el sistema y cómo se acepta.

1.2 Alcance

Firebox permitirá: - a un **vendedor**, registrar su negocio y su catálogo en un panel web y obtener un enlace de WhatsApp para su tienda; - a un **cliente**, ver el catálogo, armar un carrito, recibir la **factura electrónica validada por la DIAN** con su **QR de pago** en el mismo chat, pagar, y recibir la confirmación automática; - al **vendedor**, seguir cada pedido de *facturado a pagado* en tiempo real.

No incluye logística de envío, inventario, devoluciones ni anulación automática de facturas (ver §2.6).

1.3 Definiciones, acrónimos y abreviaturas

Término	Definición
API	Interfaz de programación de aplicaciones; aquí, el servicio FastAPI propio y las APIs externas
Bre-B	Sistema de pagos inmediatos del Banco de la República de Colombia, basado en "llaves"
CUFE	Código Único de Factura Electrónica asignado al validar ante la DIAN
DIAN	Dirección de Impuestos y Aduanas Nacionales
Factus	Proveedor de facturación electrónica (API v2)
Factus Pay	Pasarela de recaudos con QR Bre-B de Factus
Recaudo	Cobro creado en Factus Pay, con referencia y monto, que genera un QR

Término	Definición
Sandbox	Ambiente de pruebas sin validez fiscal ni movimiento de dinero real
Webhook	Llamada HTTP que un sistema externo hace a nuestra API cuando ocurre un evento
Consumidor final	Comprador que no se identifica en la factura
SDD	Spec-Driven Development: especificación → plan → tareas → código
RF / RNF	Requisito funcional / no funcional

1.4 Referencias

- Constitución del proyecto: `.specify/memory/constitution.md`.
- Especificación SDD: `specs/001-tienda-whatsapp/spec.md`.
- Integraciones y conexiones: `docs/documentacion/07_Integraciones_Conexiones.md`.
- Factus API v2: `https://developers.factus.com.co` y skill oficial `.claude/skills/facturas-crear-y-validar/SKILL.md`.
- Factus Pay: `https://pay-developers.factus.com.co` y `docs/factus-pay/FACTUS-PAY-ANALISIS-2026-09-25.md`.
- WhatsApp Cloud API: `https://developers.facebook.com/docs/whatsapp/cloud-api`.
- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.

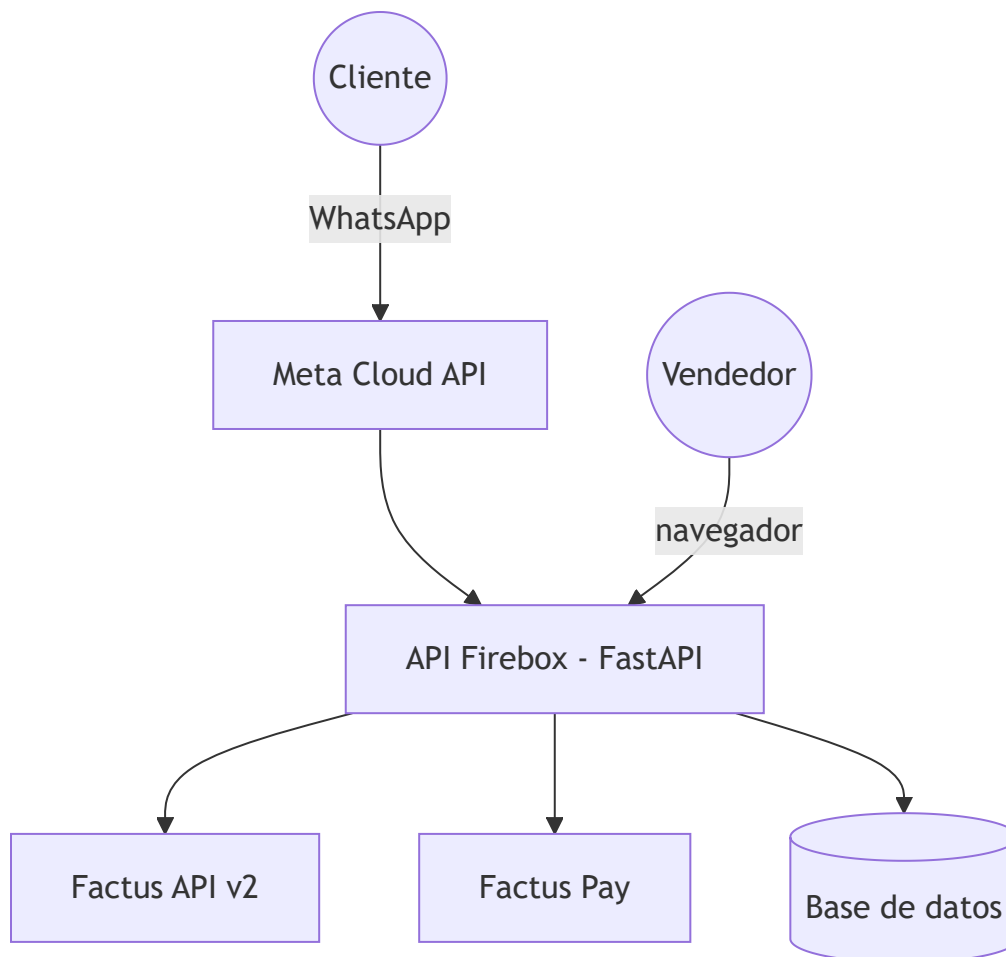
1.5 Visión general del documento

La sección 2 describe el producto en general; la sección 3 lista los requisitos específicos; la sección 4 da los criterios de aceptación; la sección 5 cruza requisitos con historias y casos de uso.

2. DESCRIPCIÓN GENERAL

2.1 Perspectiva del producto

Sistema nuevo compuesto por una aplicación propia (API + bot + panel web) y tres servicios externos:



2.2 Funciones del producto

1. Gestión de tiendas (registro, acceso, enlace de WhatsApp).
2. Gestión de catálogo (productos, categorías, IVA, fotos, carga CSV).
3. Conversación de compra en WhatsApp (catálogo, producto, carrito).
4. Facturación electrónica (datos del adquirente, emisión, validación, PDF).
5. Cobro con QR (recaudo, documento "factura + paga aquí").
6. Confirmación de pago (vigilante, mensaje al cliente).
7. Seguimiento de pedidos en vivo (panel, línea de tiempo, resumen del día).

2.3 Características de los usuarios

Usuario	Conocimiento técnico	Necesidad principal
Cliente	Usa WhatsApp; no instala nada	Comprar y pagar rápido, recibir su factura
Vendedor	Básico (navegador web)	Vender sin programar y saber quién pagó

Usuario	Conocimiento técnico	Necesidad principal
Equipo técnico	Python	Mantener e integrar

2.4 Restricciones

- WhatsApp solo por la API oficial de Meta (Cloud API v25.0); en modo de prueba, máximo 5 destinatarios.
- Facturación solo con Factus API v2 (sandbox compartido durante la hackathon).
- Cobro solo con Factus Pay (monto entre \$10.000 y \$12.000.000 por recaudo).
- Lenguaje: Python para todo (API, bot y panel web con Jinja2 + HTMX). Base de datos relacional.
- Plazo: más de 8 horas de construcción durante la hackathon.

2.5 Suposiciones y dependencias

- Las credenciales de sandbox de Factus v2 y Factus Pay siguen activas durante el evento (verificadas el 05-oct-2026).
- Meta permite crear la app y el número de prueba el día del evento.
- Hay conexión a internet estable para la demo (riesgo observado: el hotspot del líder se cayó 7 veces en 11 minutos; se usa la red de otro integrante).

2.6 Fuera de alcance (v1)

Envíos y su costo, inventario, devoluciones, nota crédito automática, otras pasarelas, otras monedas, mensajes iniciados por la tienda (plantillas), un número de WhatsApp por tienda, app móvil, varios usuarios por tienda.

3. REQUISITOS ESPECÍFICOS

3.1 Requisitos funcionales (RF)

Id	Requisito	Prioridad	Origen (spec)
RF-01	El sistema debe recibir mensajes de WhatsApp por webhook verificado con firma HMAC y responder a Meta en menos de 2 s.	Alta	FR-001.003
RF-02	El sistema debe procesar cada mensaje de WhatsApp una sola vez aunque Meta lo reenvíe.	Alta	FR-004
RF-03	El sistema debe enviar texto, listas, botones (con foto), imágenes y PDF por WhatsApp respetando los límites de Meta.	Alta	FR-005, FR-006
RF-04	El sistema debe identificar la tienda por el código <code>TIENDA-<CÓDIGO></code> o mostrar la lista de tiendas si no hay código.	Alta	FR-013

Id	Requisito	Prioridad	Origen (spec)
RF-05	El sistema debe mostrar el catálogo activo de la tienda por categorías, de 10 en 10.	Alta	FR-020
RF-06	El sistema debe mostrar cada producto con foto, nombre, precio con IVA y botones [Agregar] [Ver otro] [Ver carrito].	Alta	US1-4
RF-07	El sistema debe gestionar un carrito por cliente con cantidades de 1 a 20 y recalcular totales en cada cambio.	Alta	FR-021
RF-08	El sistema debe calcular subtotal, IVA y total por línea con redondeo bancario (half-even) y sin usar números flotantes.	Alta	FR-022
RF-09	El sistema debe validar que el total esté entre \$10.000 y \$12.000.000 antes de facturar.	Alta	FR-042
RF-10	El sistema debe permitir facturar a nombre del cliente o como consumidor final.	Alta	US2-1
RF-11	El sistema debe consultar al adquirente en la DIAN (vía Factus) para autocompletar nombre y correo, y pedirlos a mano si no existe.	Media	FR-034
RF-12	El sistema debe preguntar la forma de entrega (recoger o envío) y guardar la dirección cuando aplique.	Media	US6
RF-13	El sistema debe emitir y validar la factura electrónica en Factus v2 con referencia única por pedido.	Alta	FR-030..033
RF-14	El sistema debe comprobar que el total devuelto por Factus sea igual al total calculado; si no, detener el pedido.	Alta	FR-035
RF-15	El sistema debe crear un recaudo en Factus Pay con referencia = número de factura y monto = total de la factura.	Alta	FR-040, FR-041
RF-16	El sistema debe enviar al cliente un único PDF con la factura y una página "Paga aquí" con el QR, más la imagen del QR.	Alta	FR-050, FR-051
RF-17	El sistema debe consultar cada 5 s los cobros pendientes y marcar el pedido como pagado o vencido (24 h).	Alta	FR-043
RF-18	El sistema debe enviar una sola confirmación de pago por pedido.	Alta	FR-044
RF-19	El sistema debe permitir al cliente consultar el pago con [Ya pagué].	Media	US3-3
RF-20	El sistema debe permitir registrar una tienda validando sus credenciales de Factus Pay y entregar su enlace <code>wa.me</code> .	Alta	US4-1, US4-2
RF-21	El sistema debe permitir crear, editar, desactivar y listar productos con foto e IVA.	Alta	US4-3, US4-4
RF-22	El sistema debe permitir cargar productos desde CSV con reporte de errores por fila.	Media	US4-5

Id	Requisito	Prioridad	Origen (spec)
RF-23	El sistema debe mostrar al vendedor sus pedidos actualizados cada 5 s sin recargar.	Media	FR-062
RF-24	El sistema debe mostrar la línea de tiempo de eventos de cada pedido.	Media	FR-080
RF-25	El sistema debe permitir reintentar el cobro de un pedido en <code>error_cobro</code> .	Media	US5-3
RF-26	El sistema debe mostrar el resumen del día (pedidos, facturado, pagado).	Baja	US5-4
RF-27	El sistema debe aceptar <code>cancelar</code> , <code>menu</code> y <code>carrito</code> en cualquier paso previo a confirmar.	Media	FR-012
RF-28	(Opcional) El sistema debe buscar productos por texto libre sin que la IA fije precios.	Baja	US7

3.2 Requisitos no funcionales (RNF)

Id	Categoría	Requisito	Cómo se mide
RNF-01	Rendimiento	De [Confirmar compra] a recibir PDF y QR: ≤ 15 s en sandbox.	Cronómetro en 5 compras (SC-002)
RNF-02	Rendimiento	De pago simulado a confirmación en WhatsApp: ≤ 10 s.	SC-003
RNF-03	Exactitud	0 diferencias entre total calculado, total de Factus y monto del recaudo.	20 compras de prueba (SC-004)
RNF-04	Confiabilidad	Idempotencia: 10 webhooks iguales $\rightarrow$ 1 pedido, 1 factura, 1 recaudo.	SC-005
RNF-05	Seguridad	Credenciales cifradas (Fernet); ningún secreto en repositorio, panel, logs ni respuestas.	Revisión + búsqueda en el repositorio
RNF-06	Seguridad	Webhooks con firma inválida rechazados (401).	Prueba con firma alterada
RNF-07	Seguridad	Aislamiento multi-tienda: un token solo ve datos de su tienda.	Prueba con token ajeno
RNF-08	Usabilidad	Compra completa (1 producto, consumidor final) en ≤ 3 min y ≤ 15 toques; mensajes en español claro.	SC-001
RNF-09	Usabilidad	Registro de tienda + 3 productos en ≤ 5 min.	SC-006
RNF-10	Mantenibilidad	Canal de WhatsApp aislado en un adaptador; API documentada en OpenAPI.	Revisión de código, <code>/docs</code>
RNF-11	Portabilidad	Arranca con <code>docker-compose up</code> en cualquier equipo con Docker.	Prueba en una máquina limpia

Id	Categoría	Requisito	Cómo se mide
RNF-12	Calidad	Pruebas de reglas críticas con control negativo registrado.	SC-008
RNF-13	Compatibilidad	Toda llamada a Factus envía <code>User-Agent</code> propio.	Prueba sin User-Agent → 403

4. CRITERIOS DE ACEPTACIÓN

Un módulo se acepta cuando: 1. cumple el 100 % de sus RF de prioridad Alta y sus escenarios Dado/Cuando/Entonces de `spec.md`; 2. sus pruebas pasan y cada prueba crítica tiene control negativo (`DETECTA el bug`); 3. funciona en la URL desplegada (no solo en localhost), probado dos veces seguidas; 4. su evidencia (pantallazo o salida de comando) queda en `docs/documentacion/evidencias/`.

El sistema completo se acepta cuando se cumplen SC-001 a SC-008 de `spec.md`.

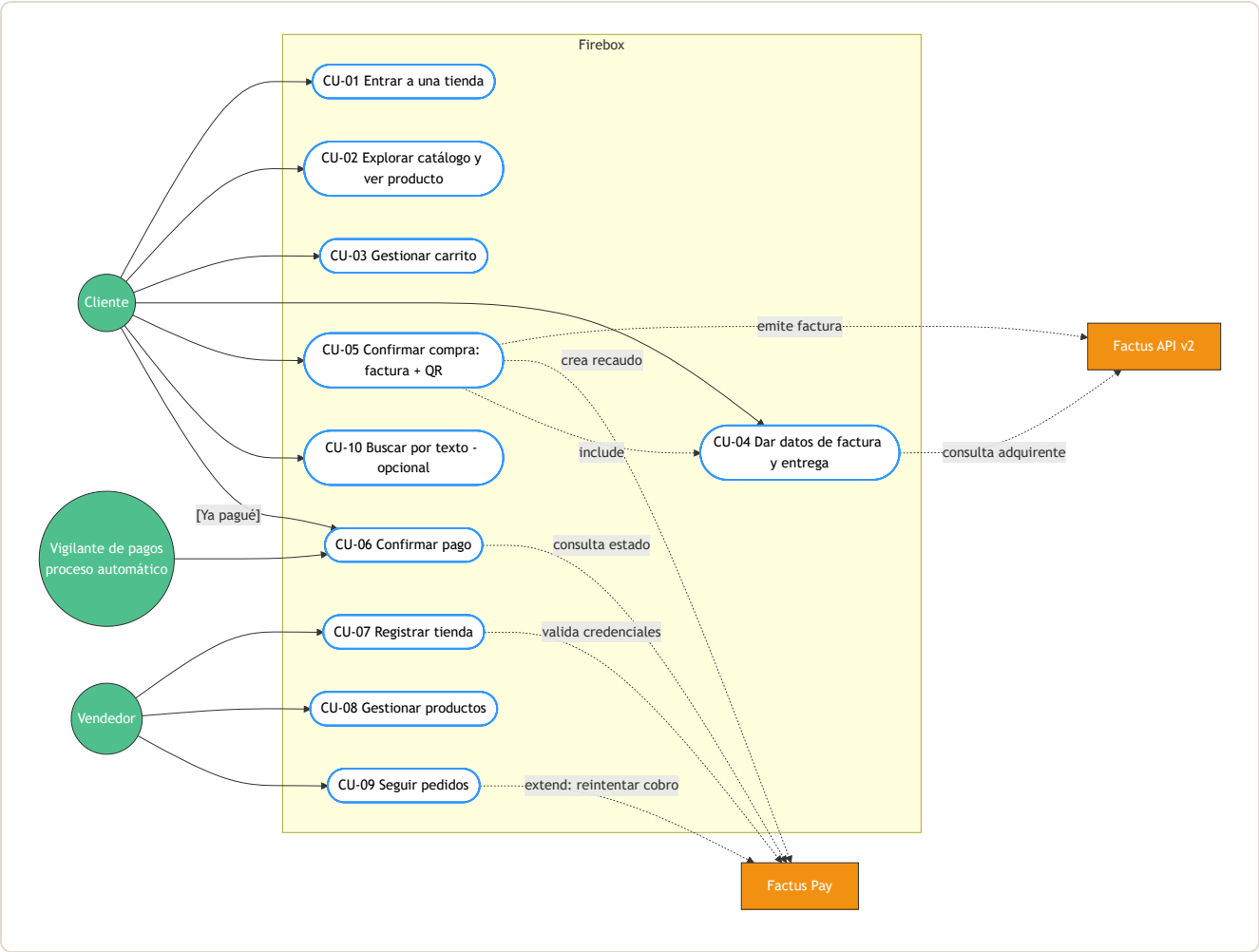
5. MATRIZ DE TRAZABILIDAD

Historia (spec)	Casos de uso (<code>02_Casos_de_Uso.md</code>)	Requisitos
US1 Comprar en el chat	CU-01, CU-02, CU-03	RF-01.08, RF-27
US2 Factura con QR	CU-04, CU-05	RF-09.16
US3 Pagar y confirmar	CU-06	RF-17.19
US4 Registrar tienda y catálogo	CU-07, CU-08	RF-20.22
US5 Seguir pedidos	CU-09	RF-23.26
US6 Entrega	CU-04	RF-12
US7 Búsqueda (opcional)	CU-10	RF-28

CASOS DE USO DEL SISTEMA

PROYECTO: Firebox - tienda multi-negocio en WhatsApp con factura electrónica y QR de pago
EQUIPO: Equipo API WARS · **FECHA:** 05-oct-2026 · **VERSIÓN:** 1.0 **FUENTE:** specs/001-tienda-whatsapp/spec.md · **Requisitos:** 01_SRS_Especificacion_Requisitos.md

1. DIAGRAMA DE CASOS DE USO



2. ACTORES

Actor	Tipo	Descripción
Cliente	Primario, humano	Compra por WhatsApp desde un número registrado (modo de prueba: máx. 5)

Actor	Tipo	Descripción
Vendedor	Primario, humano	Dueño de la tienda; usa el panel web
Vigilante de pagos	Primario, sistema	Proceso de la API que cada 5 s consulta los cobros pendientes
Factus API v2	Secundario, externo	Valida la factura ante la DIAN y entrega el PDF
Factus Pay	Secundario, externo	Crea recaudos con QR y reporta su estado
Meta Cloud API	Secundario, externo	Transporta los mensajes de WhatsApp (presente en todos los CU del cliente)

3. ESPECIFICACIÓN DE CASOS DE USO

CU-01 - Entrar a una tienda

- **Actor:** Cliente.
- **Descripción:** el cliente llega a la tienda por su enlace o código, o elige una de la lista.
- **Precondiciones:** el número del cliente está autorizado en el número de prueba de Meta; existe al menos una tienda activa.
- **Flujo principal:** 1. El cliente abre `wa.me/<número>?text=TIENDA-RELOJES` y envía el mensaje. 2. El sistema valida la firma del webhook, registra el `id` del mensaje y reconoce el código `RELOJES`. 3. El sistema guarda la tienda actual en la conversación y envía: saludo con el nombre de la tienda + botón [Ver catálogo].
- **Flujos alternos:**
 - 2a. Mensaje sin código y sin conversación previa → el sistema envía la lista de tiendas activas; el cliente elige una → paso 3.
 - 2b. Código inexistente o tienda inactiva → "No encontramos esa tienda" + lista de tiendas.
 - 2c. El cliente ya tenía carrito en otra tienda → "¿Abandonar tu carrito en «tienda»?" [Sí, cambiar] [No, seguir].
 - 2d. Mensaje repetido por Meta (`id` ya registrado) → se ignora sin responder.
- **Postcondiciones:** conversación en estado `en_tienda` con la tienda elegida.
- **Requisitos:** RF-01, RF-02, RF-04.

CU-02 - Explorar catálogo y ver producto

- **Actor:** Cliente.
- **Precondiciones:** conversación en `en_tienda` o posterior.

- **Flujo principal:** 1. El cliente pulsa **[Ver catálogo]**. 2. El sistema envía una lista interactiva con las categorías de la tienda. 3. El cliente elige una categoría; el sistema envía una lista con hasta 10 productos (título ≤ 24, descripción ≤ 72 con precio con IVA). 4. El cliente elige un producto; el sistema envía un mensaje de botones con la **foto como encabezado**, nombre, precio con IVA y **[Agregar] [Ver otro] [Ver carrito]**.
- **Flujos alternos:**
 - 2a. La tienda no tiene categorías → se salta al paso 3 con todos los productos.
 - 3a. Más de 10 productos → la última fila es "Ver más ►" (página siguiente).
 - 4a. La foto falla al enviarse → se envía el mismo contenido como texto con los botones.
- **Postcondiciones:** conversación en `viendo_producto`.
- **Requisitos:** RF-03, RF-05, RF-06.

CU-03 - Gestionar carrito

- **Actor:** Cliente.
- **Flujo principal:** 1. El cliente pulsa **[Agregar]**; el sistema pregunta la cantidad **[1] [2] [3]** (o escribir de 1 a 20). 2. El cliente elige la cantidad; el sistema agrega la línea, recalcula y confirma: "Agregado: 2 × Reloj 8314. Total parcial \$404.362". 3. El cliente pulsa **[Ver carrito]**; el sistema muestra cada línea, subtotal sin IVA, IVA y total con **[Pagar] [Seguir comprando] [Vaciar]**.
- **Flujos alternos:**
 - 1a. Cantidad escrita fuera de 1-20 o no numérica → se repite la pregunta.
 - 3a. **[Vaciar]** → carrito vacío y saludo de la tienda.
 - 3b. Producto desactivado por el vendedor mientras estaba en el carrito → se retira al ver el carrito y se avisa.
 - *. El cliente escribe `cancelar` → se vacía el carrito; `carrito` → paso 3; `menu` → saludo de la tienda.
- **Postcondiciones:** carrito persistido en la conversación con totales calculados por el motor de impuestos.
- **Requisitos:** RF-07, RF-08, RF-27.

CU-04 - Dar datos de factura y entrega

- **Actor:** Cliente. **Secundario:** Factus API v2.
- **Precondiciones:** carrito no vacío.
- **Flujo principal:** 1. El cliente pulsa **[Pagar]**; el sistema verifica que el total esté entre \$10.000 y \$12.000.000. 2. El sistema pregunta **[Factura a mi nombre] [Consumidor final]**; el cliente elige "a mi nombre". 3. El sistema pide tipo y número de documento; el cliente escribe "CC 1000000000". 4. El sistema consulta `GET /v2/dian/acquirer`; si existe, pregunta "¿Eres Juan Pérez, j@gmail.com?" * **[Sí] [Corregir]**; el cliente confirma. 5. El sistema pregunta **[Recojo en tienda] [Envío a domicilio]**; el cliente elige.
- **Flujos alternos:**

- 1a. Total fuera de rango → mensaje con el rango permitido; vuelve al carrito; no se factura.
- 2a. **[Consumidor final]** → se omiten los pasos 3-4.
- 4a. Factus responde 404 → el sistema pide nombre completo y luego correo (validado).
- 4b. **[Corregir]** → el sistema pide nombre y correo a mano.
- 5a. **[Envío a domicilio]** → el sistema pide la dirección (5-200 caracteres) y aclara que el envío se coordina por fuera.
- **Postcondiciones:** conversación en **confirmando** con datos de factura y entrega completos.
- **Requisitos:** RF-09, RF-10, RF-11, RF-12.

CU-05 - Confirmar compra: factura + QR

- **Actor:** Cliente. **Secundarios:** Factus API v2, Factus Pay.
- **Precondiciones:** CU-04 completo.
- **Flujo principal:** 1. El sistema muestra el resumen (productos, total, datos de factura, entrega) con **[Confirmar compra]** **[Cancelar]**. 2. El cliente pulsa **[Confirmar compra]**. 3. El sistema recalcula con precios vigentes, crea el pedido con copia fija de las líneas y lo pasa a **facturando**. 4. El sistema emite la factura (`POST /v2/bills/validate` , `reference_code = FB-<TIENDA>-<id>`), recibe número y CUFE, y verifica que el total de Factus sea igual al calculado. Estado **facturado**. 5. El sistema descarga el PDF de la factura. 6. El sistema crea el recaudo en Factus Pay (referencia = número de factura, monto = total) y recibe el QR. 7. El sistema arma un PDF: factura + página "Paga aquí" (tienda, número, total, QR, referencia, vencimiento 24 h, instrucciones). 8. El sistema sube y envía por WhatsApp: texto "Tu factura SETP.. está lista", el PDF, la imagen del QR y las instrucciones con **[Ya pagué]**. Estado **cobro_pendiente**.
- **Flujos alternos:**
 - 2a. Doble clic → el segundo clic encuentra el pedido en **facturando** / **facturado** y responde con el estado, sin facturar de nuevo.
 - 3a. Un precio cambió → se muestra el resumen actualizado y se pide confirmar otra vez.
 - 4a. Factus 422 → **error_factura** , mensaje amable al cliente, evento con el detalle para el panel; fin.
 - 4b. Factus 5xx o sin red → 3 reintentos (1 s, 2 s, 4 s); si persiste → 4a.
 - 4c. Total de Factus ≠ total calculado → **error_factura** ("diferencia de totales"); no se cobra.
 - 6a. Factus Pay falla tras 3 reintentos → **error_cobro** ; se envía la factura sin QR y el vendedor ve **[Reintentar cobro]**.
 - 6b. El recaudo llega con `qr: null` → se consulta `GET /v1/collections/{ref}` hasta 3 veces.
- **Postcondiciones:** pedido **cobro_pendiente** con factura validada, recaudo **ready** y documentos enviados.
- **Requisitos:** RF-13, RF-14, RF-15, RF-16.

CU-06 - Confirmar pago

- **Actores:** Vigilante de pagos (principal) y Cliente (con **[Ya pagué]**). **Secundario:** Factus Pay.

- **Precondiciones:** pedido en `cobro_pendiente`.
- **Flujo principal:** 1. Cada 5 s el vigilante consulta `GET /v1/collections/{referencia}` por cada pedido pendiente. 2. Factus Pay responde `paid`. 3. El sistema actualiza el pedido a `pagado` **solo si seguía en `cobro_pendiente`** y registra la hora. 4. Si la actualización cambió la fila, envía "✅ Pago recibido. Tu factura SETP.. quedó pagada" y registra el evento.
- **Flujos alternos:**
 - 1a. El cliente pulsa [**Ya pagué**] → la consulta se hace en ese momento; si no está pagado: "Aún no vemos el pago; puede tardar unos segundos".
 - 2a. `ready` con más de 24 h → `vencido`; se deja de consultar.
 - 3a. Otra consulta ya lo marcó `pagado` → no se envía un segundo mensaje.
 - 4a. Fuera de la ventana de 24 h de WhatsApp (error `131047`) → evento en el panel; sin reintento.
- **Postcondiciones:** pedido `pagado` (o `vencido`), cliente notificado una sola vez.
- **Requisitos:** RF-17, RF-18, RF-19.

CU-07 - Registrar tienda

- **Actor:** Vendedor. **Secundario:** Factus Pay.
- **Flujo principal:** 1. El vendedor abre el panel y elige "Crear mi tienda". 2. Ingresa nombre, NIT, correo, código corto (3-20, letras y números), contraseña del panel y correo/contraseña de Factus Pay. 3. El panel envía el formulario a la API (`POST /api/v1/tiendas`). 4. La API valida el código único, prueba las credenciales en Factus Pay (`POST /auth`), cifra las credenciales y el token, y crea la tienda. 5. La API devuelve el token del panel y el enlace `https://wa.me/<número>?text=TIENDA-<CÓDIGO>`; el panel abre la sesión del vendedor (cookie firmada) y muestra el enlace con botón de copiar.
- **Flujos alternos:**
 - 4a. Código ya usado → 409 "Ese código ya existe".
 - 4b. Credenciales de Factus Pay inválidas → 422 "No pudimos conectar con tu cuenta de Factus Pay"; no se guarda nada.
- **Postcondiciones:** tienda activa, sin productos.
- **Requisitos:** RF-20.

CU-08 - Gestionar productos

- **Actor:** Vendedor.
- **Precondiciones:** sesión iniciada (token de tienda).
- **Flujo principal:** 1. El vendedor abre "Productos" y pulsa "Nuevo". 2. Ingresa nombre, categoría, precio **sin IVA**, tarifa (19 %, 5 %, 0 % exento, excluido) y foto (JPG/PNG ≤ 5 MB). 3. La API valida, guarda y devuelve el producto con su **precio con IVA** calculado. 4. El panel lo muestra en la lista.
- **Flujos alternos:**
 - 2a. Foto > 5 MB o de otro tipo → rechazo en el panel.
 - 3a. Precio ≤ 0 o tarifa inválida → 422 con el campo.

- A. Editar → `PATCH` ; los pedidos anteriores conservan su copia fija.
- B. Desactivar → deja de verse en WhatsApp.
- C. Importar CSV → la API crea las filas válidas y devuelve el reporte de las rechazadas.
- **Postcondiciones:** catálogo actualizado y visible en WhatsApp de inmediato.
- **Requisitos:** RF-21, RF-22.

CU-09 - Seguir pedidos

- **Actor:** Vendedor. **Secundario:** Factus Pay (en reintento de cobro).
- **Flujo principal:** 1. El vendedor abre "Pedidos"; la página (HTMX) pide la lista de pedidos cada 5 s. 2. El panel muestra cada pedido con fecha, cliente, total, estado (color), número de factura y enlace al PDF. 3. El vendedor abre un pedido y ve la línea de tiempo de eventos.
- **Flujos alternos:**
- 3a. Pedido en `error_cobro` → **[Reintentar cobro]** → `POST /api/v1/pedidos/{id}/reintentar-cobro` → si funciona, el cliente recibe el QR.
- 3b. Pedido en `error_factura` → el panel muestra el detalle de Factus para corregir (p. ej. el producto).
- **Postcondiciones:** ninguna (consulta), salvo el reintento.
- **Requisitos:** RF-23, RF-24, RF-25, RF-26.

CU-10 - Buscar por texto (opcional)

- **Actor:** Cliente.
- **Flujo principal:** el cliente escribe "reloj dorado mujer"; el sistema busca por nombre, categoría y color y responde con una lista de hasta 10 productos con precios **leídos de la base de datos**.
- **Flujo alternativo:** sin resultados → "No encontré eso" + **[Ver catálogo]**.
- **Requisitos:** RF-28.

4. CONCLUSIÓN

Los diez casos de uso cubren las siete historias de `spec.md` y sus 28 requisitos funcionales. Los casos CU-05 y CU-06 concentran las tres integraciones externas y los candados de idempotencia; por eso sus flujos alternos están descritos con el mayor detalle.

HISTORIAS DE USUARIO

PROYECTO: Firebox · **EQUIPO:** Equipo API WARS · **FECHA:** 05-oct-2026 · **VERSIÓN:** 1.0 **FUENTE:** specs/001-tienda-whatsapp/spec.md (los criterios completos Dado/Cuando/Entonces están allí)

1. FORMATO

Cada historia sigue: **Como** «rol» **quiero** «acción» **para** «beneficio». Se estima en horas de trabajo de la hackathon y se asigna a una de las cuatro líneas de trabajo:

Línea	Qué construye	Perfil
L1 Canal y conversación	Webhook de Meta, adaptador de envío, máquina de estados del chat	Python
L2 Facturación y documento	Motor de impuestos, cliente Factus v2, PDF "factura + paga aquí"	Python
L3 Pagos y vigilante	Cliente Factus Pay, vigilante, idempotencia, reintentos	Python
L4 Panel	Pantallas del vendedor: plantillas Jinja2 + HTMX servidas por la misma FastAPI	Python + HTML

La API REST del panel (endpoints de tiendas, productos y pedidos) la construye L3 junto con el modelo de datos, porque es la línea con menos trabajo de integración al inicio.

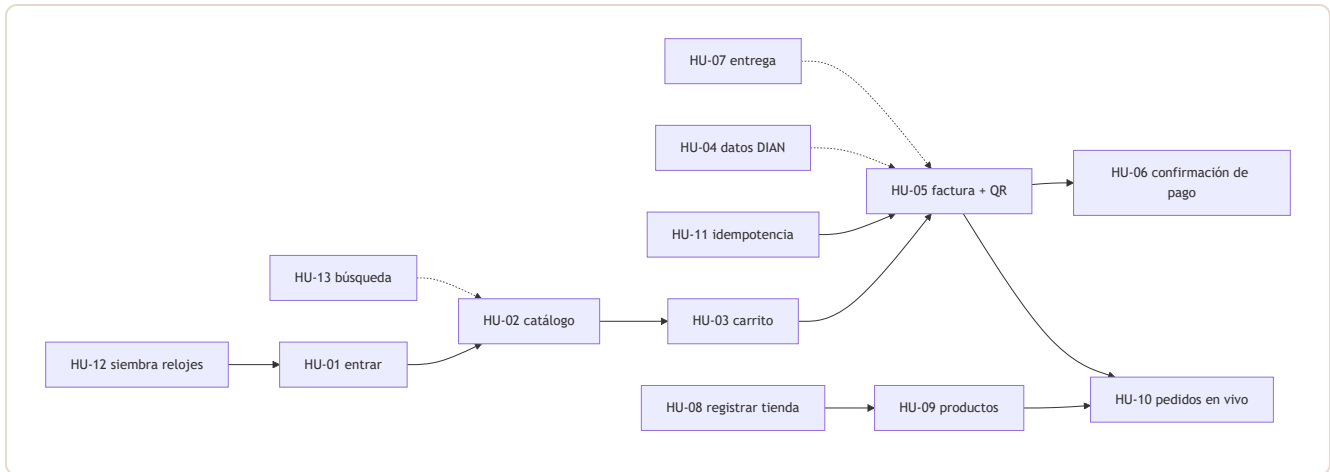
2. HISTORIAS

Id	Historia	Prioridad	Estimación	Línea	Criterios clave (resumen)
HU-01	Como cliente quiero entrar a una tienda con su enlace para comprar ahí sin buscarla	Alta	1 h	L1	TIENDA-RELOJES → saludo + [Ver catálogo]; sin código → lista de tiendas
HU-02	Como cliente quiero ver el catálogo por categorías con fotos y precios con IVA para elegir qué comprar	Alta	2 h	L1	Listas ≤ 10 filas con "Ver más"; tarjeta con foto + 3 botones
HU-03	Como cliente quiero un carrito con totales exactos para saber cuánto voy a pagar	Alta	2 h	L1 + L2	Cantidad 1-20; subtotal, IVA, total half-even; [Pagar] [Seguir] [Vaciar]
HU-04	Como cliente quiero facturar a mi nombre sin escribir todo para terminar rápido	Media	1,5 h	L2	Consulta DIAN vía Factus; 404 → datos a mano; consumidor final

Id	Historia	Prioridad	Estimación	Línea	Criterios clave (resumen)
HU-05	Como cliente quiero recibir mi factura electrónica con el QR de pago en el chat para tener soporte y pagar en un paso	Alta	3 h	L2 + L3	Factura validada (CUFE), recaudo ref = número factura, PDF combinado + PNG del QR
HU-06	Como cliente quiero que me confirmen el pago solos para no mandar comprobantes	Alta	2 h	L3	≤ 10 s tras pagar; un solo mensaje; [Ya pagué]; vencido a 24 h
HU-07	Como cliente quiero decir si recojo o me envían para que el negocio sepa qué hacer	Media	0,5 h	L1	Dirección 5-200 caracteres; envío sin costo en v1
HU-08	Como vendedor quiero registrar mi tienda para tener mi enlace de WhatsApp	Alta	2 h	L3 + L4	Valida Factus Pay; cifra credenciales; devuelve token + enlace <code>wa.me</code>
HU-09	Como vendedor quiero cargar mis productos con foto e IVA para que aparezcan en WhatsApp	Alta	2,5 h	L3 + L4	CRUD; precio con IVA calculado por la API; desactivar; CSV (P2)
HU-10	Como vendedor quiero ver mis pedidos cambiar a pagado en vivo para saber qué despachar	Media	2 h	L4	HTMX cada 5 s; línea de tiempo; [Reintentar cobro]; resumen del día
HU-11	Como equipo queremos que un mensaje repetido no cree dos facturas para no duplicar documentos fiscales	Alta	1 h	L1 + L3	10 webhooks iguales → 1 pedido, 1 factura, 1 recaudo
HU-12	Como equipo queremos cargar los 74 relojes de NovaMarket para tener una tienda real en la demo	Alta	1 h	L3	Script de siembra desde <code>catalogo_tienda.json</code> con fotos
HU-13	Como cliente quiero buscar escribiendo para no recorrer todo el catálogo	Baja	1,5 h	L1	Opcional; precios siempre de la base de datos

Total estimado: 22,5 h de trabajo ≈ 5,5-6 h de reloj con 4 personas en paralelo, más integración, despliegue y pitch.

3. ORDEN DE ENTREGA (camino crítico de la demo)



Línea continua = imprescindible para la demo. Línea punteada = suma valor pero la demo funciona sin ella.

4. DEFINICIÓN DE TERMINADO (para cada historia)

1. Cumple sus escenarios de `spec.md`.
2. Tiene pruebas, y las de reglas críticas con control negativo (`DETECTA el bug`).
3. Revisada por otro integrante (pull request a `main`).
4. Funciona en el ambiente desplegado.
5. Evidencia guardada en `docs/documentacion/evidencias/<HU>/` (pantallazo o salida del comando).

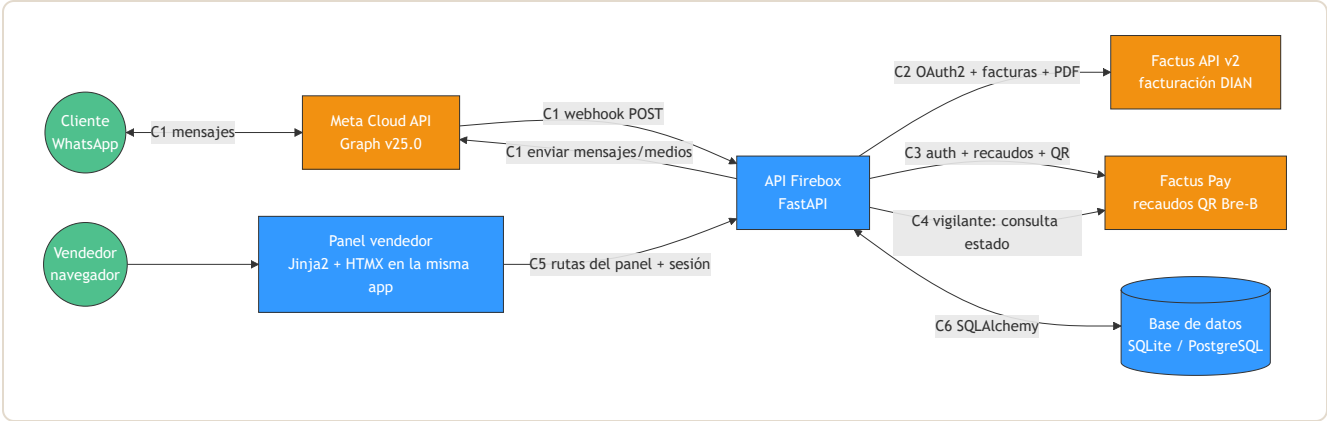
DOCUMENTACIÓN DE INTEGRACIONES Y CONEXIONES DEL SISTEMA

PROYECTO: Firebox - tienda multi-negocio en WhatsApp con factura electrónica (Factus) y QR de pago (Factus Pay) **EQUIPO:** Equipo API WARS (integrantes en `RET0.md`) · Líder técnico: Fernando Vega Benavides **EVENTO:** API WARS Hackathon 2026 - Universidad Distrital Francisco José de Caldas, Facultad Tecnológica **FECHA:** 05-oct-2026 · **VERSIÓN:** 1.0 **FUENTE DE REQUISITOS:** `specs/001-tienda-whatsapp/spec.md` · **REGLAS:** `.specify/memory/constitution.md`

1. INTRODUCCIÓN

Este documento describe, conexión por conexión, cómo se comunica Firebox con cada sistema: qué hace cada conexión, con qué credenciales, qué endpoints usa, qué envía y qué recibe, qué errores puede devolver, qué límites tiene, y **qué se verificó en vivo** contra los ambientes de prueba el 05-oct-2026. Cada dato marcado ☒ fue medido; cada dato marcado ☐ viene de la documentación oficial y se verifica durante la implementación.

2. MAPA GENERAL DE CONEXIONES



Id	Conexión	Dirección	Protocolo	Autenticación
C 1	WhatsApp (Meta Cloud API)	entrada (webhook) y salida	HTTPS JSON	Firma HMAC <code>X-Hub-Signature-256</code> (entrada) · Bearer token de sistema (salida)
C 2	Factus API v2	salida	HTTPS, form-urlencoded (token) y JSON	OAuth2 <code>password</code> + <code>refresh_token</code>

Id	Conexión	Dirección	Protocolo	Autenticación
C3	Factus Pay (crear recaudo)	salida	HTTPS JSON	POST /auth → token Bearer que no vence
C4	Factus Pay (vigilante de pagos)	salida, periódica	HTTPS JSON	el mismo token de C3
C5	Panel web → servicios de la API	entrada	HTTPS (HTML + fragmentos HTMX) y REST JSON	Cookie de sesión firmada (panel) · Bearer token de tienda (REST)
C6	API → base de datos	interna	SQL (SQLAlchemy 2)	usuario/contraseña en DATABASE_URL

3. VARIABLES DE ENTORNO POR CONEXIÓN

Conexión	Variables (en .env, nunca en el repositorio)
C1	META_GRAPH_BASE_URL=https://graph.facebook.com , META_GRAPH_API_VERSION=v25.0 , META_PHONE_NUMBER_ID , META_ACCESS_TOKEN , META_APP_SECRET , META_VERIFY_TOKEN , WHATSAPP_NUMERO_PUBLICO
C2	FACTUS_BASE_URL , FACTUS_CLIENT_ID , FACTUS_CLIENT_SECRET , FACTUS_USERNAME , FACTUS_PASSWORD (respaldo v1: FACTUS_V1_*)
C3/C4	FACTUS_PAY_BASE_URL (+ credenciales por tienda cifradas en la tabla tiendas ; para sembrar: FACTUS_PAY_EMAIL/PASSWORD , FACTUS_PAY_PERSONAL_*)
C5	SESSION_SECRET (firma de la cookie de sesión del panel)
C6	DATABASE_URL (sqlite:///./firebox.db local · postgresql+psycopg://... desplegado)
Transversal	FERNET_KEY (cifrado de credenciales por tienda), APP_USER_AGENT=Firebox/1.0

Comprobación rápida de C2 y C3: `python scripts/verificar_credenciales.py` (solo tokens y lecturas; con control negativo probado).

4. C1 - WHATSAPP (META CLOUD API)

4.1 Propósito

Recibir los mensajes del cliente (texto, botón pulsado, fila de lista elegida) y enviarle el catálogo, el carrito, la factura en PDF, el QR y las confirmaciones.

4.2 Entrada: webhook

Paso	Método y ruta	Qué hace
Verificación (una vez, al configurar la app en Meta)	GET /webhooks/whatsapp? hub.mode=subscribe&hub.verify_token= hub.challenge=...	Si <code>hub.verify_token</code> = <code>META_VERIFY_TOKEN</code> , responder 200 con el valor de <code>hub.challenge</code> en texto plano; si no, 403
Mensajes	POST /webhooks/whatsapp	1) validar firma, 2) responder 200 en < 2 s, 3) procesar en segundo plano

Validación de firma (FR-002): `X-Hub-Signature-256: sha256=<hex>`; calcular `HMAC-SHA256(META_APP_SECRET, cuerpo_crudo)` y comparar en tiempo constante (`hmac.compare_digest`). Usar el cuerpo **crudo**, antes de parsear el JSON.

Forma del POST que envía Meta  (se extrae lo necesario):

```
{
  "object": "whatsapp_business_account",
  "entry": [{
    "changes": [{
      "field": "messages",
      "value": {
        "metadata": { "phone_number_id": "<id del número>" },
        "contacts": [{ "wa_id": "57300XXXXXX", "profile": { "name": "Juan" } }],
        "messages": [{
          "id": "wamid.XXXX",
          "from": "57300XXXXXX",
          "timestamp": "1759670000",
          "type": "interactive",
          "interactive": { "type": "button_reply", "button_reply": { "id": "agregar:REL-8314",
"message_id": "57300XXXXXX" } } },
          "title": "Agregar" } }
        ],
        "statuses": []
      }
    ]
  }
}
```

<code>messages[].type</code>	Dónde viene lo que eligió el cliente
<code>text</code>	<code>text.body</code>
<code>interactive</code> + <code>button_reply</code>	<code>interactive.button_reply.id</code>
<code>interactive</code> + <code>list_reply</code>	<code>interactive.list_reply.id</code>

Los `statuses` (entregado, leído, fallido) se registran como eventos; **no** generan respuesta.

Idempotencia (FR-004): antes de procesar, insertar `messages[].id` en `mensajes_procesados` (clave única). Si ya existe, terminar.

Convención de id de botones y filas (≤ 256 caracteres botón, ≤ 200 fila): `<acción>:<dato>`, por ejemplo `tienda:RELOJES`, `cat:HOMBRE`, `prod:REL-8314`, `agregar:REL-8314`, `cant:2`, `pagar`, `factura:propia`, `factura:cf`, `confirmar:<id pedido>`.

4.3 Salida: enviar mensajes

POST `{META_GRAPH_BASE_URL}/{META_GRAPH_API_VERSION}/{META_PHONE_NUMBER_ID}/messages` con `Authorization: Bearer {META_ACCESS_TOKEN}`.

Tipo	Cuerpo (resumen)	Límites  (doc. oficial leída el 05-oct)
Texto	<code>{"type":"text","text":{"body":"..."}}</code>	4096 caracteres
Lista	<code>{"type":"interactive","interactive":{"type":"list","header":...,"body":...,"action":{"button":"Ver","sections":[{"title":...,"rows":[{"id","title","description"}]}]}}</code>	máx. 10 secciones y 10 filas en total ; header 60, body 4096, footer 60, botón 20, título de sección 24, título de fila 24, descripción de fila 72 , id de fila 200
Botones	<code>{"type":"interactive","interactive":{"type":"button","header":{"type":"image","image":{"link":...},"body":...,"action":{"buttons":[{"type":"reply","reply":{"id","title"}}]}}</code>	máx. 3 botones , título 20, id 256, body 1024, footer 60; header puede ser text, image , video o document
Imagen	<code>{"type":"image","image":{"link":"https://..."}}</code> o <code>{"id":"<media_id>"}</code>	JPG/PNG hasta 5 MB (CRM de Vexon)
Documento	<code>{"type":"document","document":{"id":"<media_id>","filename":"Factura-SETP...pdf","caption":"..."}}</code>	hasta 100 MB (CRM de Vexon)

Todo cuerpo lleva además `"messaging_product":"whatsapp"`, `"recipient_type":"individual"`, `"to":"<wa_id>"`.

Subir un medio propio (el PDF combinado y el PNG del QR, que no tienen URL pública): POST `{versión}/{phone_number_id}/media` `multipart/form-data` con `file`, `type` (`application/pdf` o `image/png`) y `messaging_product=whatsapp` → devuelve `{"id":"<media_id>"}`. Patrón ya implementado en `vexon-crm-codigo/src/server/whatsapp/media.ts` (referencia).

4.4 Errores de Meta que se manejan

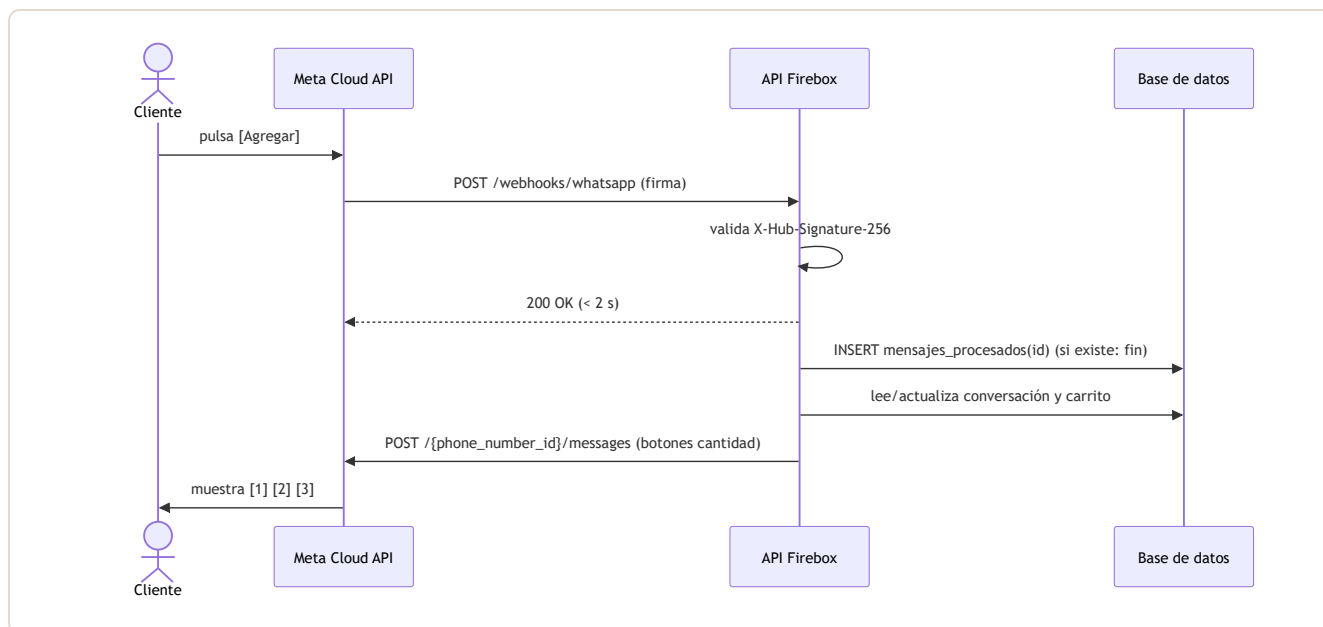
Código	Significado	Qué hace Firebox
131030	Destinatario no está en la lista permitida del número de prueba	Evento <code>destinatario_no_autorizado</code> ; aviso en panel

Código	Significado	Qué hace Firebox
131047	Pasaron más de 24 h desde el último mensaje del cliente	Evento <code>fuera_de_ventana</code> ; no reintenta
130429 / 131056	Límite de envío	Reintento con espera creciente (1 s, 2 s, 4 s)
190	Token vencido o inválido	Error crítico en logs; la demo se detiene hasta cambiar <code>META_ACCESS_TOKEN</code>

4.5 Restricciones del modo de prueba

- Número de prueba de Meta: solo envía a **5 destinatarios** registrados en la app (cada uno confirma con un código).
- Mensajes iniciados por la tienda requieren **plantillas** aprobadas: fuera de alcance; todo el flujo responde dentro de la ventana de 24 h que abre el cliente.

4.6 Secuencia C1



5. C2 - FACTUS API v2 (FACTURA ELECTRÓNICA)

5.1 Propósito

Emitir y validar ante la DIAN la factura electrónica de cada pedido, consultar datos del adquirente y descargar el PDF.

5.2 Datos de conexión (medidos el 05-oct-2026)

Dato	Valor
URL base sandbox	https://api-sandbox.factus.com.co
Cuenta	sandbox v2 compartida, entregada por la organización (sandboxv2@...) - responde 200 en /v2/* y 403 en /v1/*
Rango de numeración de facturas	numbering_range_id = 389 , prefijo SETP , documento "Factura de Venta", activo
Facturas ya existentes en la cuenta	23.155 (última SETP990023156) : otros equipos usan la misma cuenta
Token	expires_in = 3600 s, trae refresh_token
User-Agent	obligatorio . Control negativo: sin User-Agent propio → 403 de Cloudflare (error 1010) ; con Firebox/1.0 → 200
Límite	 80 solicitudes/min por cuenta; 429 con Retry-After

5.3 Endpoints usados

#	Método y ruta	Para qué	Respuesta relevante
1	POST /oauth/token (form-urlencoded: grant_type=password, client_id, client_secret, username, password)	Token	access_token, refresh_token, expires_in, token_type 
2	POST /oauth/token (grant_type=refresh_token, client_id, client_secret, refresh_token)+ Authorization: Bearer <token actual>	Renovar	igual a 1
3	GET /v2/numbering-ranges	Hallar el rango de Factura de Venta activo (al arrancar)	id=389 , prefix=SETP 
4	GET /v2/dian/acquirer? identification_document_code=13&identification_number=<n>	Autocompletar nombre y correo	200 con datos, o 404 {"message": "No se encontró el adquirente"} 
5	POST /v2/bills/validate (JSON)	Emitir y validar la factura	201 con data.bill (número, CUFE, totales, is_validated , errors) 
6	GET /v2/bills/{numero}/download-pdf	PDF de la factura	data.file_name , data.pdf_base_64_encoded  (≈ 69 KB en base64)

#	Método y ruta	Para qué	Respuesta relevante
7	GET /v2/bills/{numero}	Ver una factura (panel, reintentos)	data.number, reference_code, is_validated, errors, customer, ... 
8	DELETE /v2/bills/destroy/reference/{reference_code}	Borrar una factura no validada que quedó a medias	

Cabeceras de toda llamada: Accept: application/json , Authorization: Bearer <access_token> , User-Agent: Firebox/1.0 .

5.4 Cuerpo de la factura (FR-032), construido por la API

```
{
  "numbering_range_id": 389,
  "document": "01",
  "operation_type": "10",
  "reference_code": "FB-RELOJES-000123",
  "observation": "Pedido por WhatsApp en Firebox - tienda RELOJES",
  "payment_details": [
    { "payment_form": "2", "payment_method_code": "47", "amount": "202181.00", "due_date": "2026-10-06" }
  ],
  "customer": {
    "identification_document_code": "13",
    "identification": "1000000000",
    "names": "Juan Pérez",
    "email": "juan@example.com",
    "legal_organization_code": "2",
    "tribute_code": "ZZ",
    "municipality_code": "11001"
  },
  "items": [
    {
      "code_reference": "REL-8314",
      "name": "Reloj Curren 8314 negro",
      "quantity": "1.00",
      "discount_rate": "0.00",
      "price": "169900.00",
      "unit_measure_code": "94",
      "standard_code": "999",
      "taxes": [ { "code": "01", "rate": "19.00" } ]
    }
  ]
}
```

Reglas del cuerpo (fuente: skill oficial `facturas-crear-y-validar` + memoria del proyecto Factus Nova + ejemplos oficiales):

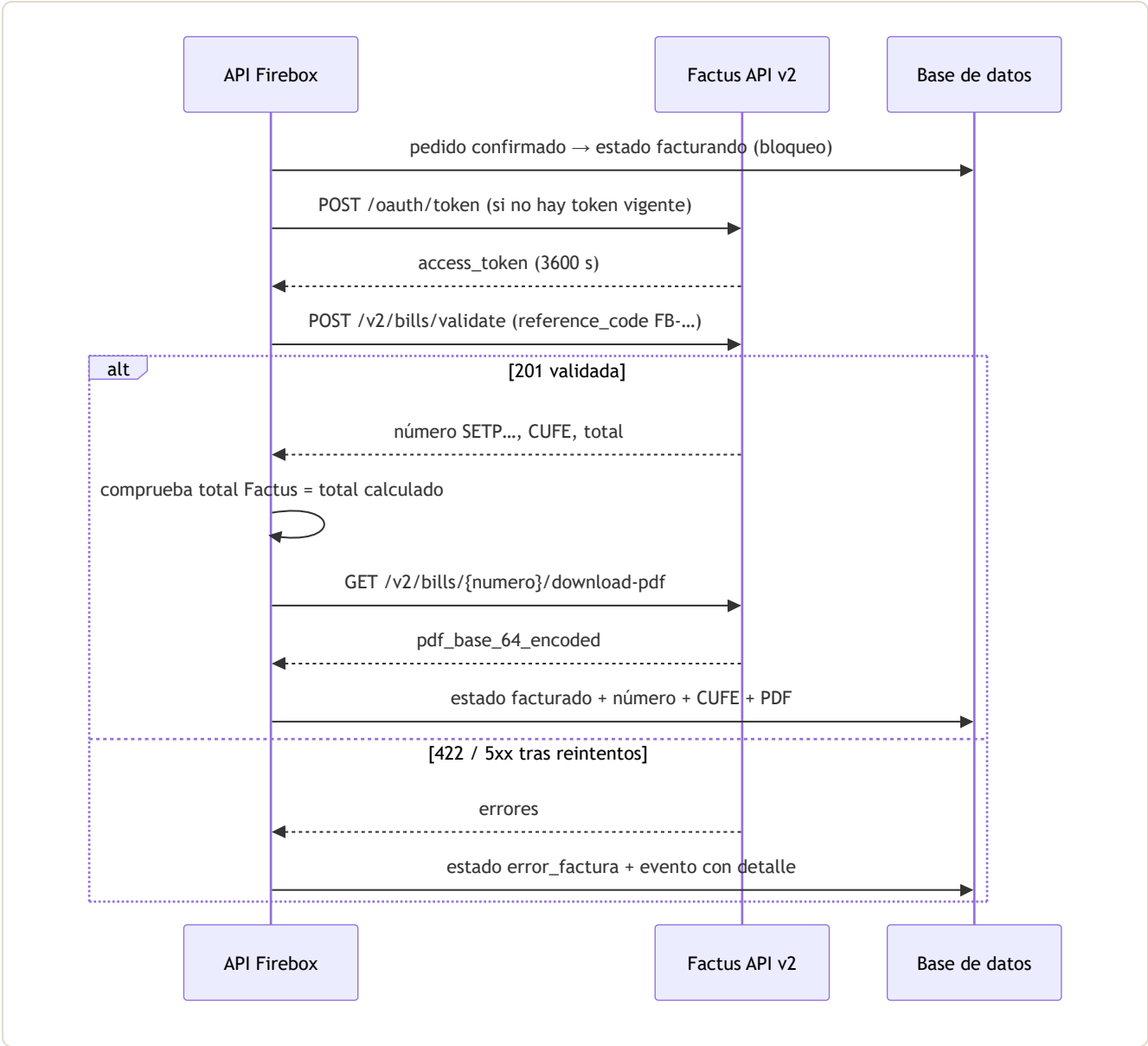
- `price` va **sin impuestos** (en v1 era con impuestos). Números como **string** con máximo 2 decimales; `numbering_range_id` como entero.
- `unit_measure_code` = "94" (unidad). El "70" **no existe** y produce 422 (bug ya visto en Factus Nova).
- `payment_details[].amount` es **obligatorio** (sin él → 422).
- Excluido: `{"code":"01","rate":"0.00","is_excluded":true}`; exento: `rate "0.00"` sin `is_excluded`.
- Consumidor final: usar exactamente el `customer` del ejemplo oficial "Con consumidor final": `identification_document_code "13"`, `identification "2222222222"`, `names "Consumidor Final"`.
- Forma de pago: crédito ("2") con vencimiento al día siguiente, porque el pago llega **después** de la factura (decisión del equipo: factura con QR, luego pago). Ejemplo oficial "A crédito" usa `payment_method_code "47"`. 🔍 **Pendiente de verificar en implementación:** que el sandbox acepte crédito + consumidor final; si lo rechaza, usar `payment_form "1"` (contado) con `payment_method_code "47"` y dejarlo anotado aquí.
- `reference_code` con prefijo `FB-<TIENDA>` para no chocar con otros equipos en la cuenta compartida.

5.5 Errores de Factus que se manejan

HTTP	Causa típica	Qué hace Firebox
401	Token vencido	Renovar (endpoint 2) y reintentar una vez
403 "Version de API no disponible para esta empresa"	Credenciales de la versión equivocada (v1 en v2 o al revés)	Error de configuración: revisar <code>.env</code>
403 Cloudflare 1010	Falta <code>User-Agent</code>	Error de programación: el cliente HTTP siempre lo envía
409 / 422 por <code>reference_code</code> repetido	Pedido ya facturado	Consultar la factura existente (endpoint 7) y continuar con ella
422	Datos inválidos (<code>errors</code> por campo)	<code>error_factura</code> + detalle en el panel; no se cobra
429	Límite de solicitudes	Esperar <code>Retry-After</code> y reintentar
5xx / sin red	Factus caído	3 reintentos con espera 1 s, 2 s, 4 s; luego <code>error_factura</code>

Las **notificaciones** DIAN en **errors** (FAK08, FAJ44b, RUT01...) **no** invalidan la factura si **is_validated = true**.

5.6 Secuencia C2



6. C3 - FACTUS PAY (CREAR EL COBRO CON QR)

6.1 Propósito

Crear un recaudo por el total de la factura y obtener el QR Bre-B que el cliente escanea para pagar.

6.2 Datos de conexión (✅ medidos el 25-sep y re-verificados el 05-oct-2026)

Dato	Valor
URL base sandbox	<code>https://pay-api-sandbox.factus.com.co</code> (producción: <code>https://pay-api.factus.com.co</code> , sale a mediados de oct-2026)
Cuentas	equipo (enviada por <code>retofactus@halltec.co</code>) y personal de Fernando: ambas ✅ token y listado 200 el 05-oct
Token	<code>POST /auth</code> → <code>{token}</code> ; no vence (respuesta escrita de Factus, 26-sep)
Límites	<code>/auth</code> 5 por minuto ✅; <code>/v1/*</code> 80 por minuto ✅
Rango de monto	10.000 a 12.000.000 COP ✅ (9.999 y 12.000.001 → 422)

6.3 Endpoints usados

#	Método y ruta	Cuerpo	Respuesta medida ✅
1	<code>POST /auth</code>	<code>{"email", "password"}</code>	<code>{"token": "..."} (≈ 51 caracteres)</code>
2	<code>POST /v1/collections</code>	<code>{"reference_code": "SETP990023200", "amount": 202181}</code>	200 <code>{"data": {"reference_code", "amount", "status": "ready", "created_at", "qr": "data:image/png;base64,..."}, "status": "success", "message": "Recaudo asignado y QR generado correctamente"}</code> (la doc dice 201/ <code>started / qr:null</code> : aceptar ambos)
3	<code>GET /v1/collections/{reference_code}</code>	-	detalle con <code>status</code> y <code>qr</code> ; 404 "Recaudo no encontrado"
4	<code>GET /v1/collections?status=&reference_code=</code>	-	lista paginada de 15

Comportamientos medidos que importan: - Repetir `reference_code` devuelve **el mismo recaudo** e **ignora el monto nuevo** → idempotente, pero hay que garantizar que el monto correcto vaya en el **primer** intento. - El QR viene como **data-URI PNG**: se decodifica el base64 para el PDF y para enviarlo como imagen por WhatsApp. - El QR del sandbox es formato Mono (`mono:breb-participant:sandbox:qr:...`): solo lo lee el **simulador**; en producción será el estándar de llaves Bre-B a nombre de "Factus Pay". - No hay webhook, ni cancelación, ni datos del pagador por API.

6.4 Errores

HTTP	Causa	Qué hace Firebox
401 Unauthenticated .	Token inválido (cambiaron la contraseña)	Re-autenticar una vez (respetando 5/min)
422	Monto fuera de rango o referencia > 100	No debería ocurrir (se valida antes, FR-042); si ocurre → <code>error_cobro</code>
429	Límite	Esperar y reintentar
5xx / "Error... en Mono"	Falla del proveedor	3 reintentos; luego <code>error_cobro</code> y botón [Reintentar cobro] en el panel

7. C4 - VIGILANTE DE PAGOS (FACTUS PAY, CONSULTA PERIÓDICA)

7.1 Propósito

Factus Pay **no avisa** cuando entra un pago (no hay webhook; Factus lo confirmó el 26-sep: "endpoint de consulta de estado por referencia para hacer el proceso de polling"). El vigilante pregunta por cada cobro pendiente.

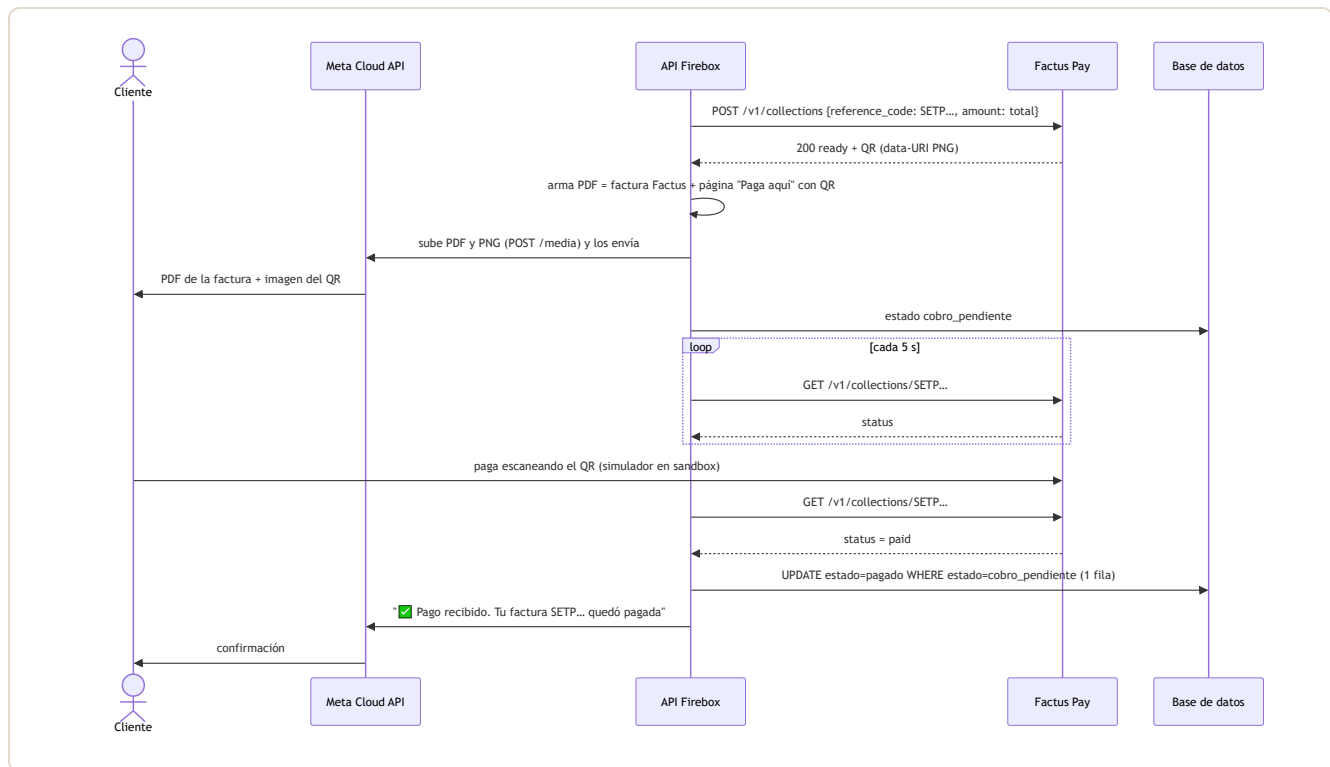
7.2 Funcionamiento

- Tarea en segundo plano dentro del servicio FastAPI (arranca con la app), ciclo cada **5 s**.
- Toma los pedidos en `cobro_pendiente` agrupados por tienda (cada tienda tiene su cuenta y su límite de 80/min).
- `GET /v1/collections/{referencia}` :
- `paid` → transición **idempotente** a `pagado` (`UPDATE ... WHERE estado = 'cobro_pendiente'`); solo si la fila cambió se envía el mensaje de confirmación por C1.
- `ready` y > 24 h desde la creación → `vencido` .
- Error de red → se reintenta en el siguiente ciclo.
- El cliente también puede forzar una consulta con **[Ya pagué]** (misma lógica, misma idempotencia).

7.3 Cómo se simula el pago en la demo

Panel sandbox `https://pay-api-sandbox.factus.com.co/simulator` : se escanea el QR con la cámara, o se automatiza con Playwright (entrar en `/login` con `button[type=submit]` , ir a `/simulator` y ejecutar `window.Livewire.all()[0].$wire.call('handleScannedQr', <texto del QR> )`). Guion probado: `referencias/factus-pay-scripts/simular_pago.py` . ⚠ Los tres escenarios de fallo del simulador (saldo, cuenta, timeout) **no funcionan** en el sandbox ("Error... en Mono").

7.4 Secuencia C3 + C4 (cobro y confirmación)



8. C5 - PANEL WEB Y API REST PROPIA

8.1 Propósito

El vendedor administra su tienda desde un panel web que vive **en la misma aplicación FastAPI** (decisión del equipo del 05-oct: nadie del equipo maneja Laravel, así que todo queda en Python). El panel no tiene lógica propia: sus pantallas llaman a las **mismas funciones de servicio** que la API REST, así una pantalla y un endpoint nunca pueden calcular distinto (Constitución II).

8.2 Reglas

- **Dos puertas, una lógica:**
- **Panel** (`/panel/ ...`): HTML con plantillas **Jinja2**; las partes que cambian (lista de pedidos, resumen) son fragmentos que **HTMX** pide cada 5 s (`hx-get="/panel/pedidos/tabla" hx-trigger="every 5s"`) sin recargar la página.
- **API REST** (`/api/v1/ ...`): JSON, documentada sola en `/docs` (OpenAPI). Sirve para pruebas, para el jurado y para integrar otros sistemas en el futuro.
- **Autenticación:**
- Panel: inicio de sesión con correo y contraseña del vendedor → **cookie de sesión firmada** (`SESSION_SECRET`), `httpOnly` , `SameSite=Lax` ; los formularios llevan token CSRF.

- API REST: `Authorization: Bearer <token de tienda>` (el token se entrega al registrar la tienda y se guarda solo como hash).
- **Errores:** la API responde `{"detail": "...", "code": "..."}` con 400/401/404/409/422; el panel muestra `detail` en el formulario.

8.3 Endpoints REST previstos (el contrato exacto se escribe en `specs/001-tienda-whatsapp/contracts/api-panel.yaml` en la fase de plan; cada pantalla del panel usa el mismo servicio que su endpoint)

Método y ruta	Para qué	Historia
<code>POST /api/v1/tiendas</code>	Registrar tienda (valida credenciales Factus Pay) → token + enlace <code>wa.me</code>	4
<code>POST /api/v1/sesiones</code>	Iniciar sesión del vendedor → token	4
<code>GET /api/v1/tienda</code>	Datos de mi tienda (sin secretos)	4
<code>GET/POST /api/v1/productos</code> , <code>PATCH/DELETE /api/v1/productos/{id}</code>	CRUD de productos (precio con IVA calculado por la API)	4
<code>POST /api/v1/productos/importar</code>	Carga CSV con reporte por fila	4
<code>GET /api/v1/pedidos?estado=&desde=</code>	Lista de pedidos	5
<code>GET /api/v1/pedidos/{id}</code>	Detalle + línea de tiempo de eventos	5
<code>POST /api/v1/pedidos/{id}/reintentar-cobro</code>	Reintento en <code>error_cobro</code>	5
<code>GET /api/v1/pedidos/{id}/pdf</code>	Descargar el PDF combinado	5
<code>GET /api/v1/resumen?fecha=</code>	Pedidos, facturado y pagado del día	5

9. C6 - API → BASE DE DATOS

- SQLAlchemy 2 con un solo `DATABASE_URL` : SQLite en desarrollo (cero instalación para el equipo), PostgreSQL en el despliegue.
- Migraciones con Alembic. Claves únicas que sostienen la idempotencia: `mensajes_procesados.id` , `pedidos.reference_code_factura` , `pedidos.numero_factura` , `tiendas.codigo` .
- El modelo detallado (entidad-relación, normalización y diccionario de datos) se documenta en `04_Modelo_Datos.md` (fase de plan).

10. CÓMO SE PRUEBA CADA CONEXIÓN

Conexión	Prueba automática	Prueba real	Control negativo
C1	Webhook con payload grabado + firma calculada	Mensaje desde un teléfono registrado	Firma alterada → debe dar 401
C2	Cliente Factus contra respuestas grabadas	1 factura real en sandbox v2	Quitar <code>User-Agent</code> → debe fallar con 403
C3	Cliente Factus Pay contra respuestas grabadas	1 recaudo real en sandbox	Monto 9.999 → la API debe frenarlo antes de facturar
C4	Vigilante con reloj simulado	Pago con el simulador	Dos <code>paid</code> seguidos → 1 solo mensaje
C5	Pruebas de endpoints con <code>TestClient</code>	Panel desplegado con Playwright	Token de otra tienda → 404/401, nunca datos ajenos
C6	Migraciones en SQLite y PostgreSQL	-	Insertar mensaje duplicado → debe chocar la clave única

`scripts/verificar_credenciales.py` cubre hoy C2 y C3 (control negativo hecho: contraseñas dañadas → 400/401 y salida 1).

11. REGISTRO DE DIFERENCIAS ENTRE DOCUMENTACIÓN Y REALIDAD

Fecha	Servicio	Documentación dice	Realidad medida
25-sep-2026	Factus Pay	Crear recaudo → 201, <code>started</code> , <code>qr: null</code>	200, <code>ready</code> , QR ya generado
25-sep-2026	Factus Pay	Colección Postman con cabecera <code>X-Factus-Access-Key</code>	La API no la exige
25-sep-2026	Factus Pay	Simulador con 3 escenarios de fallo	Los 3 responden "Error... en Mono"
05-oct-2026	Factus v2	(no lo menciona)	Sin <code>User-Agent</code> propio → 403 Cloudflare 1010
05-oct-2026	Factus v2	Cuenta sandbox	Compartida: 23.155 facturas de otros usuarios

12. CONCLUSIÓN

Firebox depende de tres servicios externos y los tres ya respondieron a nuestras credenciales el 05-oct-2026 (Factus v2 y Factus Pay medidos; Meta Cloud API en uso en otros proyectos del líder técnico, con número de prueba por configurar). Los riesgos conocidos (User-Agent, cuenta compartida, falta de webhook de pago, límite de 5 destinatarios) tienen una respuesta de diseño documentada arriba.

PARTE III

Pruebas y evidencias

Evidencias medidas de cada tarea: corridas reales, registros y capturas.

Evidencia Fase 0 - corrida real de punta a punta (05-oct-2026, 12:46)

Comando: `python app/demo_corte_vertical.py --para 57313XXXXXXX --espera 8` (número del cliente enmascarado). Pago simulado por Fernando en el simulador del sandbox de Factus Pay; los 4 mensajes y la confirmación llegaron a su WhatsApp.

```
[ 0.0 s] Carrito: 1 x Reloj Curren 8314 negro | subtotal $169.900 + IVA $32.281 = TOTAL $202.181
[ 5.3 s] Factura SETP990023179 emitida | validada DIAN: True | CUFE: c0480313a5de09b6...
[ 6.1 s] PDF de la factura descargado (50 KB)
[ 8.8 s] Recaudo en Factus Pay: referencia SETP990023179 | $202.181 | estado ready
[ 9.0 s] Documento único (factura + Paga aquí) guardado en app\salida\Factura-SETP990023179.pdf
[ 17.2 s] Factura, PDF y QR enviados por WhatsApp
[ 17.2 s] Vigilando el pago (cada 5 s, hasta 8 min). Paga el QR en el simulador de Factus Pay...
[ 257.7 s]  PAGADO: factura SETP990023179
[ 258.8 s] Confirmación enviada por WhatsApp
```

Medido: factura + recaudo en 8,8 s; entrega en WhatsApp a los 17,2 s (meta SC-002: 15 s, falta optimizar el envío de medios); confirmación enviada 1,1 s después de detectar el pago.

Evidencia gráfica: panel vendedor

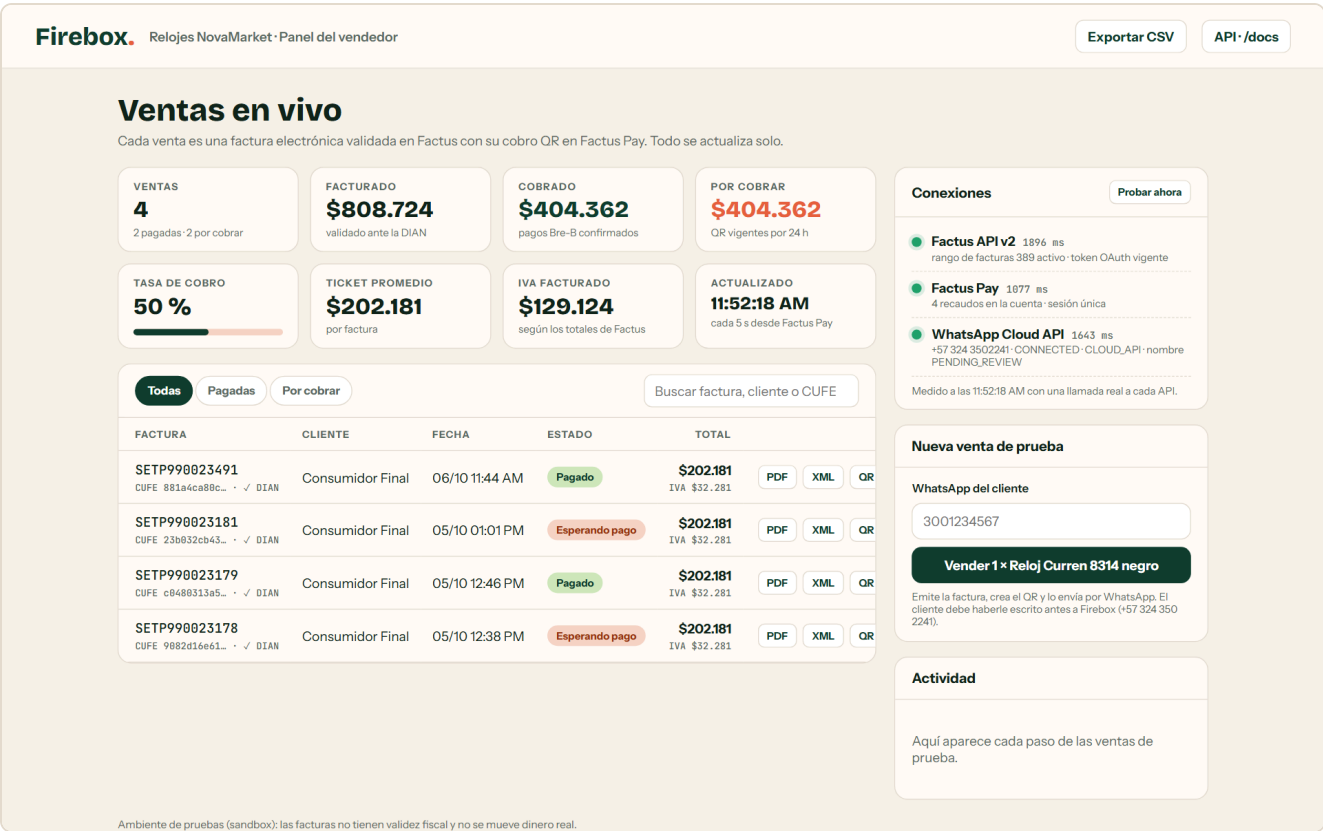


Figura. panel vendedor (archivo docs/documentacion/evidencias/panel/panel_vendedor.png).

Conclusiones

Este documento reúne, en un solo volumen, la documentación técnica de Firebox: una tienda dentro de WhatsApp que emite la factura electrónica con Factus API v2 y la cobra con un QR de Factus Pay, administrada desde un panel web. Su contenido es el de los archivos fuente del repositorio; no se añadieron cifras ni afirmaciones nuevas.

- El trabajo se organizó con Spec-Driven Development: constitución, especificación, plan, modelo de datos, contratos y tareas preceden al código (Parte I).
- El análisis y diseño se documentó con requisitos, casos de uso, historias de usuario e integraciones externas medidas (Parte II).
- Las pruebas quedan respaldadas por evidencias fechadas en el repositorio (Parte III).
- Las facturas del sandbox de Factus no tienen validez fiscal ante la DIAN, y Factus Pay se usó en su entorno de pruebas.

Capítulos incluidos en esta edición

1. Introducción y el reto
2. Constitución del proyecto – Firebox (API WARS 2026)
3. Feature Specification: Firebox – tienda en WhatsApp con factura electrónica y QR de pago (001-tienda-whatsapp)
4. DOCUMENTO DE ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE (SRS)
5. CASOS DE USO DEL SISTEMA
6. HISTORIAS DE USUARIO
7. DOCUMENTACIÓN DE INTEGRACIONES Y CONEXIONES DEL SISTEMA
8. Evidencia Fase 0 – corrida real de punta a punta (05-oct-2026, 12:46)
9. Evidencia gráfica: panel vendedor

Referencias

1. Factus. Documentación de la API de facturación electrónica. <https://developers.factus.com.co>
2. Factus Pay. Documentación para desarrolladores. <https://pay-developers.factus.com.co>
3. Meta. WhatsApp Cloud API. <https://developers.facebook.com/docs/whatsapp/cloud-api>
4. GitHub. Spec Kit (Spec-Driven Development). <https://github.com/github/spec-kit>
5. IEEE. IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
6. FastAPI. Documentación oficial. <https://fastapi.tiangolo.com>
7. HTMX. Documentación oficial. <https://htmx.org>
8. Python Software Foundation. Módulo `decimal` de la biblioteca estándar.
<https://docs.python.org/3/library/decimal.html>